QUT Centre for Robotics



ENN583 – Foundations of Robotic Vision

Week 8 – Sem 2 – 2026

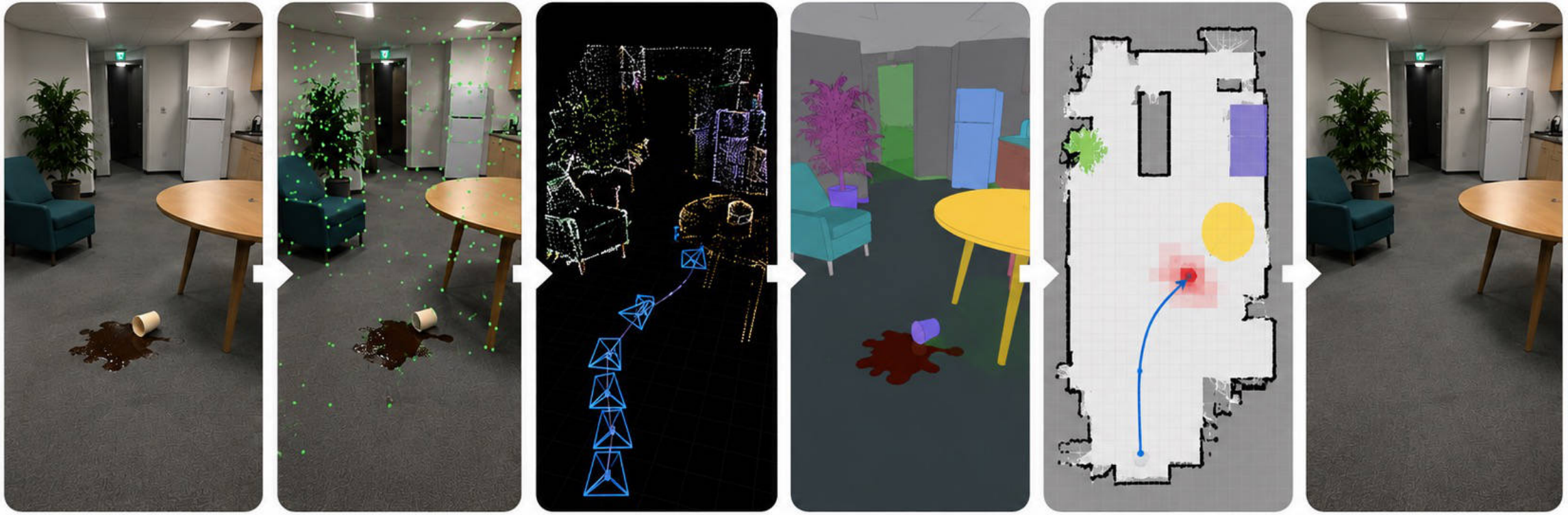
Prof Niko Suenderhauf

Welcome to Week 8!



CRICOS No.00213J

What is Robotic Vision?

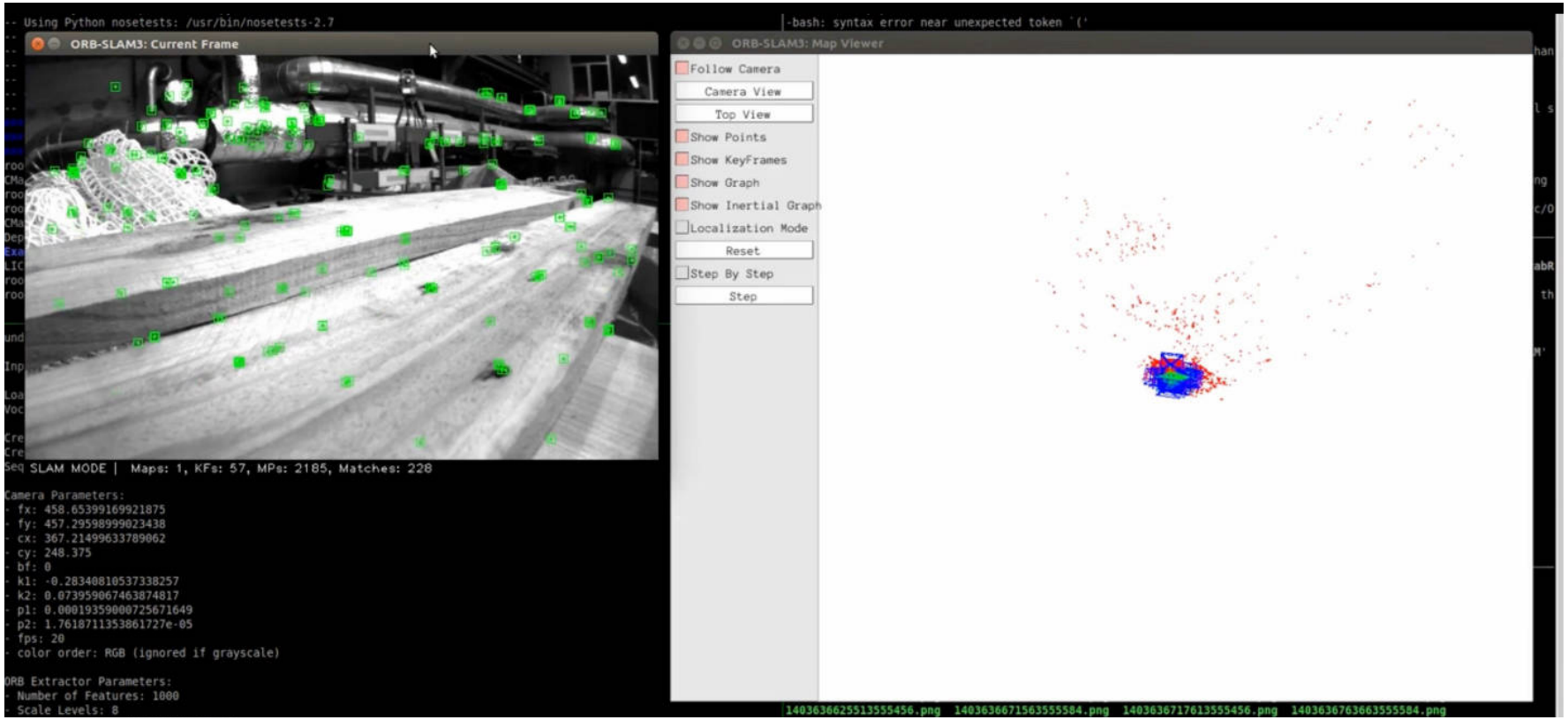


From pixels to information to actions.

CRICOS No.00213J

Example 1 – Visual Odometry

We can estimate camera motion and 3D structure of the world by matching visual evidence across images.

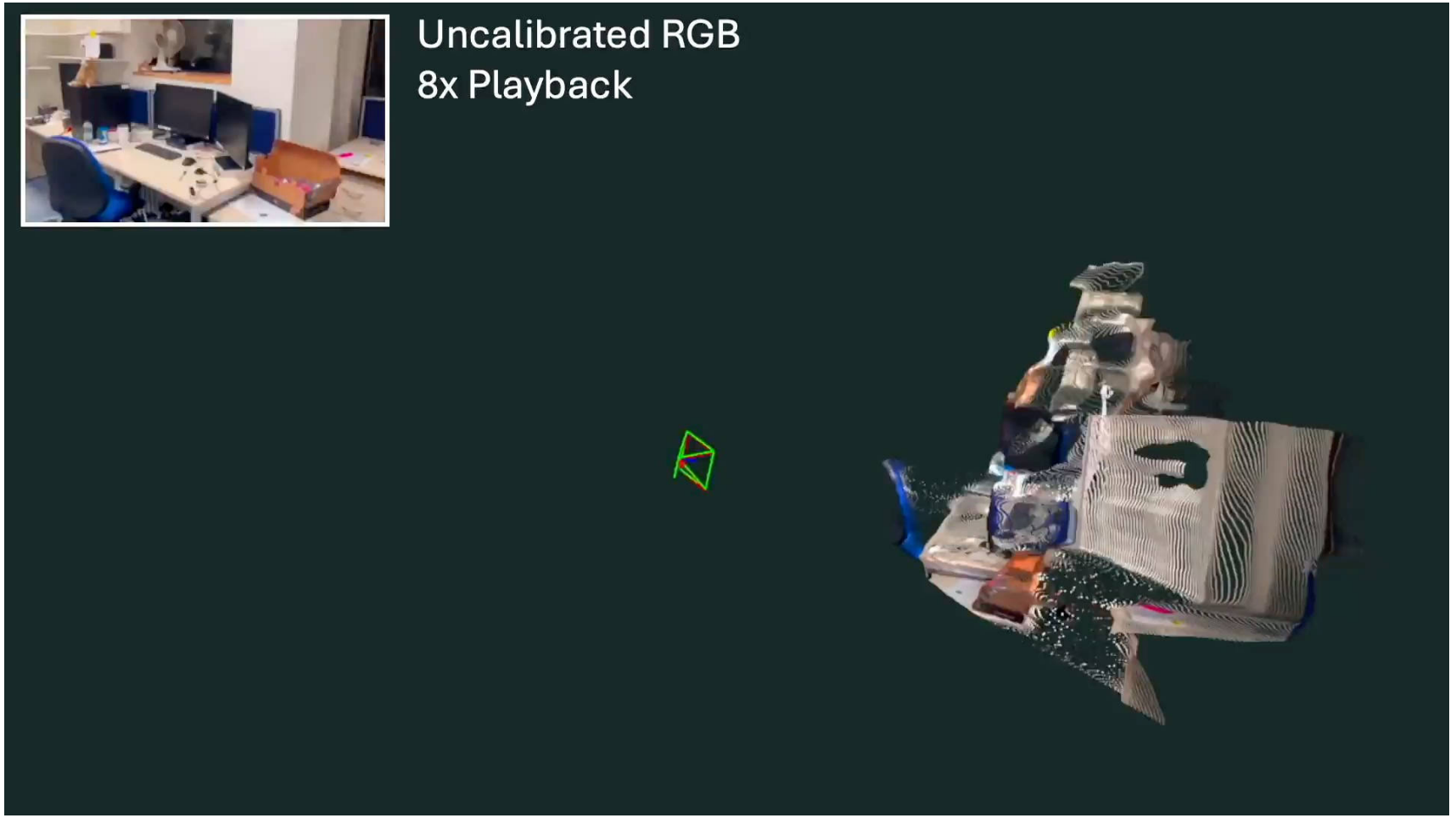


CRICOS No.00213J

ORB-SLAM3 Demo on EuRoC MH01 dataset, https://www.youtube.com/watch?v=WfY6mCTnFGg

Example 2 – Visual SLAM

A robot can build a consistent map of the world by aggregating motion information and correcting estimation errors.



CRICOS No.00213J

Riku Murai, Eric Dexheimer, Andrew J. Davison. MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors. CVPR 2025.

We covered that in previous weeks, but there is more!

CRICOS No.00213J

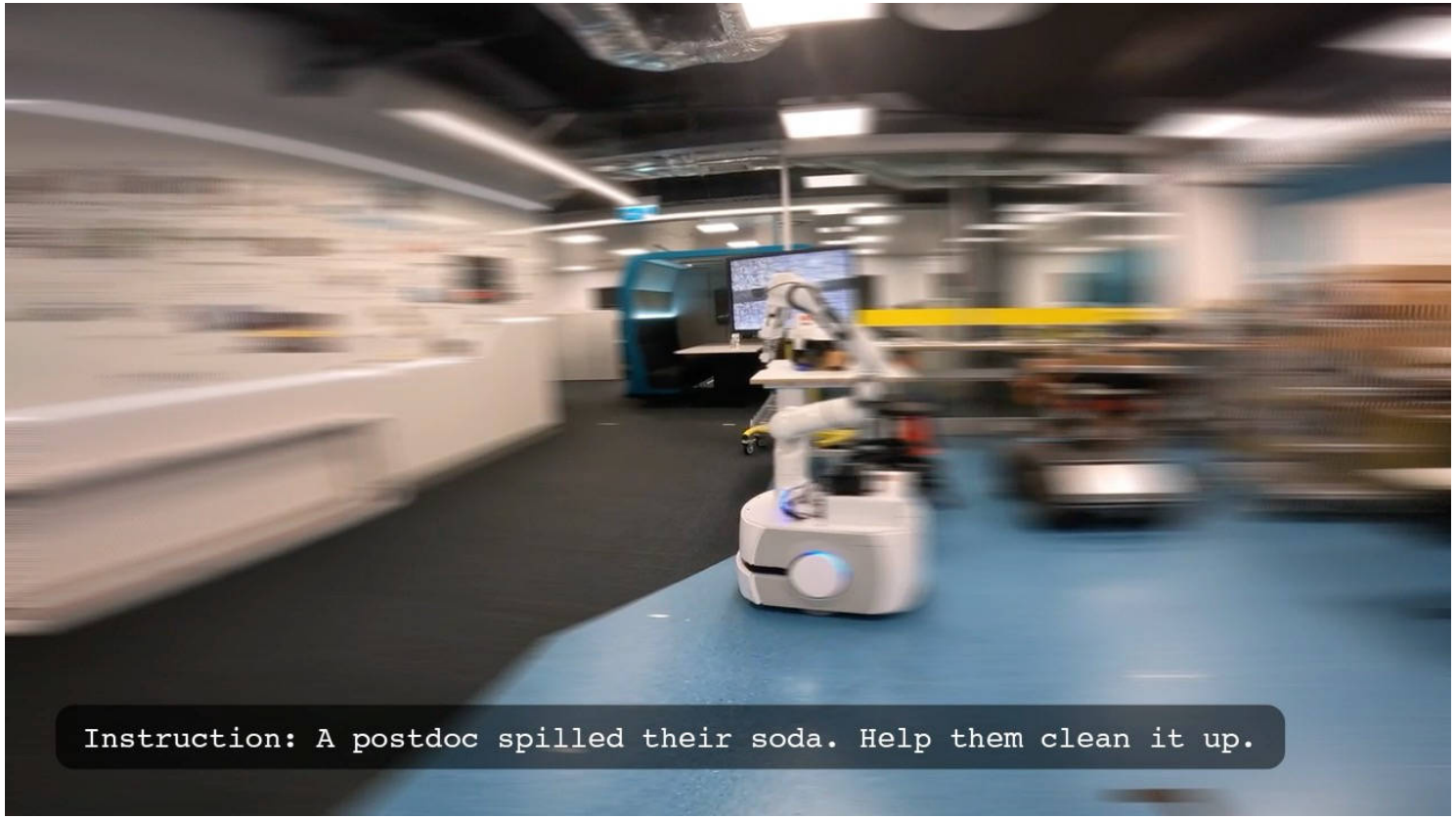
What does this robot need to …

… see?

… know or remember?

… do?

… avoid getting wrong?



Some tasks are harder than others

- We can solve some problems using **intuition** and **expert knowledge** and create a **hand-crafted solution**.

CRICOS No.00213J

Some tasks are harder than others

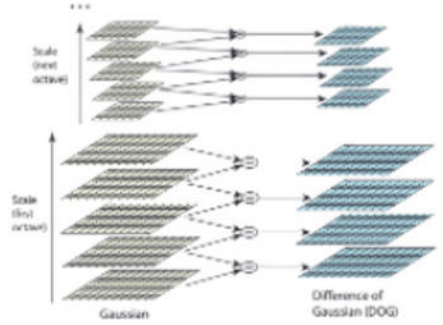
- We can solve some problems using **intuition** and **expert knowledge** and create a **hand-crafted solution**.
  - Example: Simple interest point detectors and descriptors (Week 2)

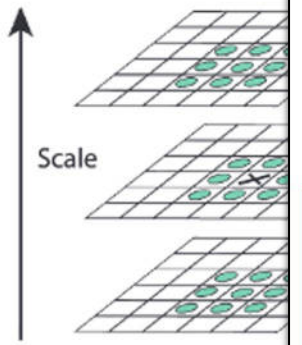
SIFT – Scale Space Idea

Find candidate feature points as local maxima or minima by comparing each pixel with its neighbours in the same image, and in the scale above and below.

A candidate must be locally distinctive in both **space and scale**.

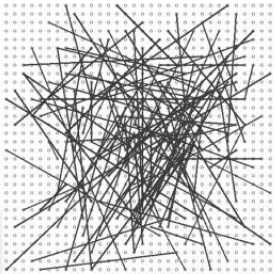






BRIEF

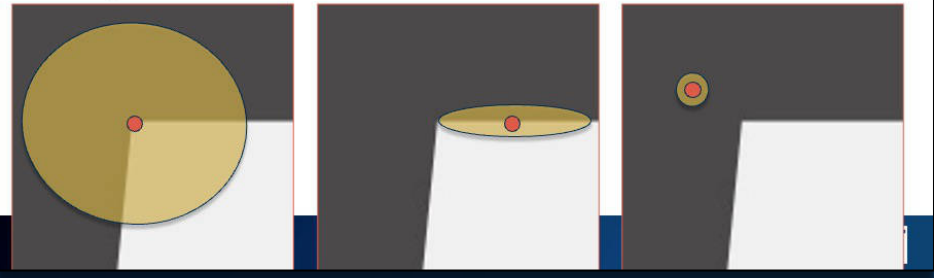
- Describes a keypoint by intensity comparisons between pairs of pixels in a patch



Structure Tensor: Key Property

$$A = w * \begin{pmatrix} J_u^2 & J_u J_v \\ J_u J_v & J_v^2 \end{pmatrix}$$

- **Two large eigenvalues**: the pixel is at a **corner**, where intensity changes significantly in both directions.
- **One large, one small eigenvalue**: the pixel lies on an **edge**, where intensity changes in one direction only.
- **Two small eigenvalues**: the pixel is in a **flat region** with little to no intensity change.



Some tasks are harder than others

- We can solve some problems using **intuition** and **expert knowledge** and create a **hand-crafted solution**.

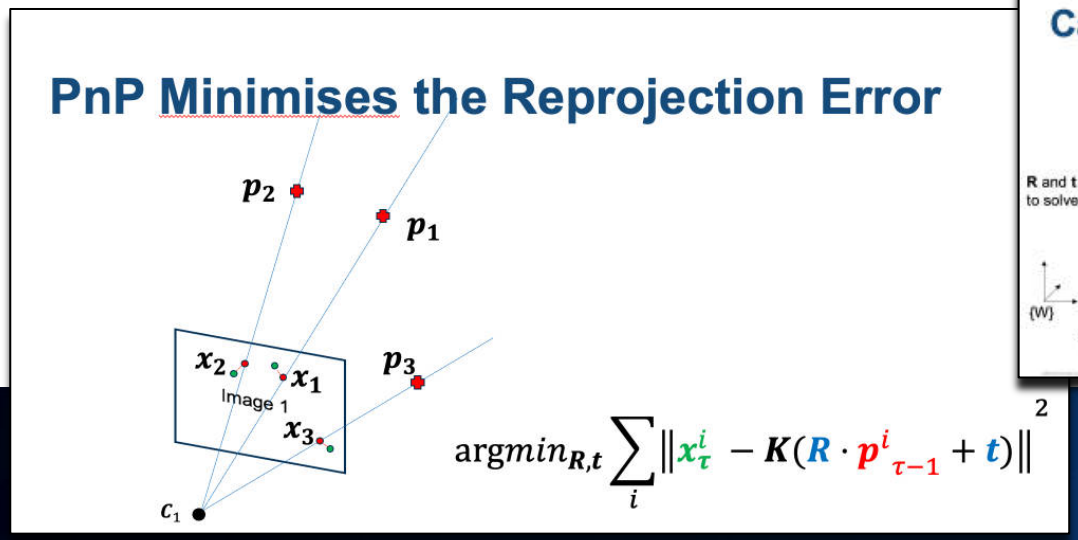
- For others, we can’t find a solution easily, but can write a ”score” function that **evaluates a potential solution**. We can then solve via **optimisation**.

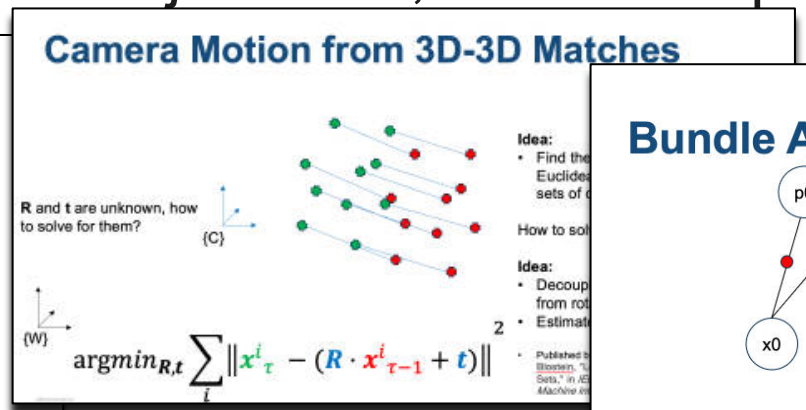
CRICOS No.00213J

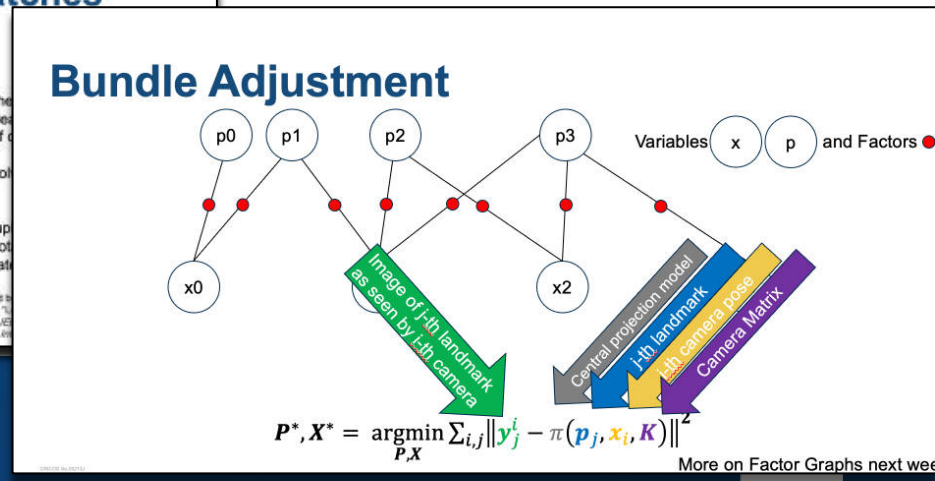
Some tasks are harder than others

- We can solve some problems using **intuition** and **expert knowledge** and create a **hand-crafted solution**.

- For others, we can't find a solution easily, but can write a "score" function that **evaluates a potential solution**. We can then solve via **optimisation**.
  - Example: Solving PnP, Bundle Adjustment, Factor Graphs







Some tasks are harder than others

- We can solve some problems using **intuition** and **expert knowledge** and create a **hand-crafted solution**.

- For others, we can't find a solution easily, but can write a "score" function that **evaluates a potential solution**. We can then solve via **optimisation**.

- For other problems we can't even write a simple "scoring" function, but we might be able to **learn a solution from data.**

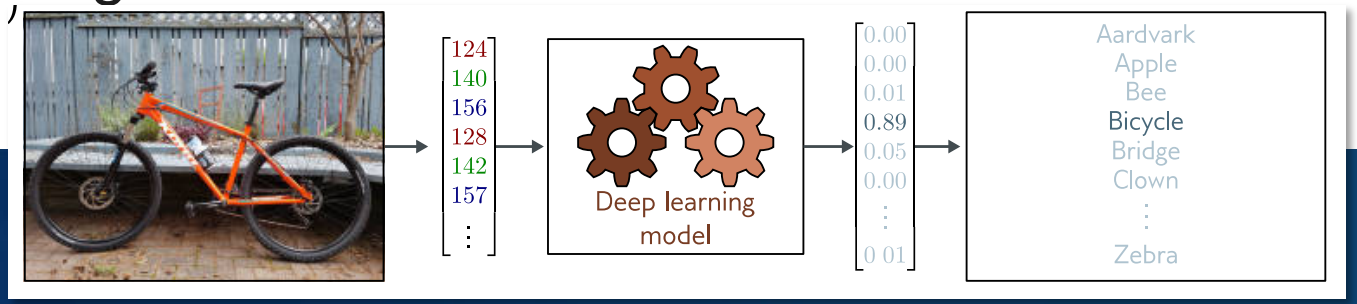
CRICOS No.00213J

Some tasks are harder than others

- We can solve some problems using **intuition** and **expert knowledge** and create a **hand-crafted solution**.

- For others, we can't find a solution easily, but can write a "score" function that **evaluates a potential solution**. We can then solve via **optimisation**.

- For other problems we can't even write a simple "scoring" function, but we might be able to **learn a solution from data.**
  - **Machine Learning!** Example: Image classification



Example – Scene Understanding

Robots can understand the world around them by using neural networks that are trained to detect objects and other relevant information.



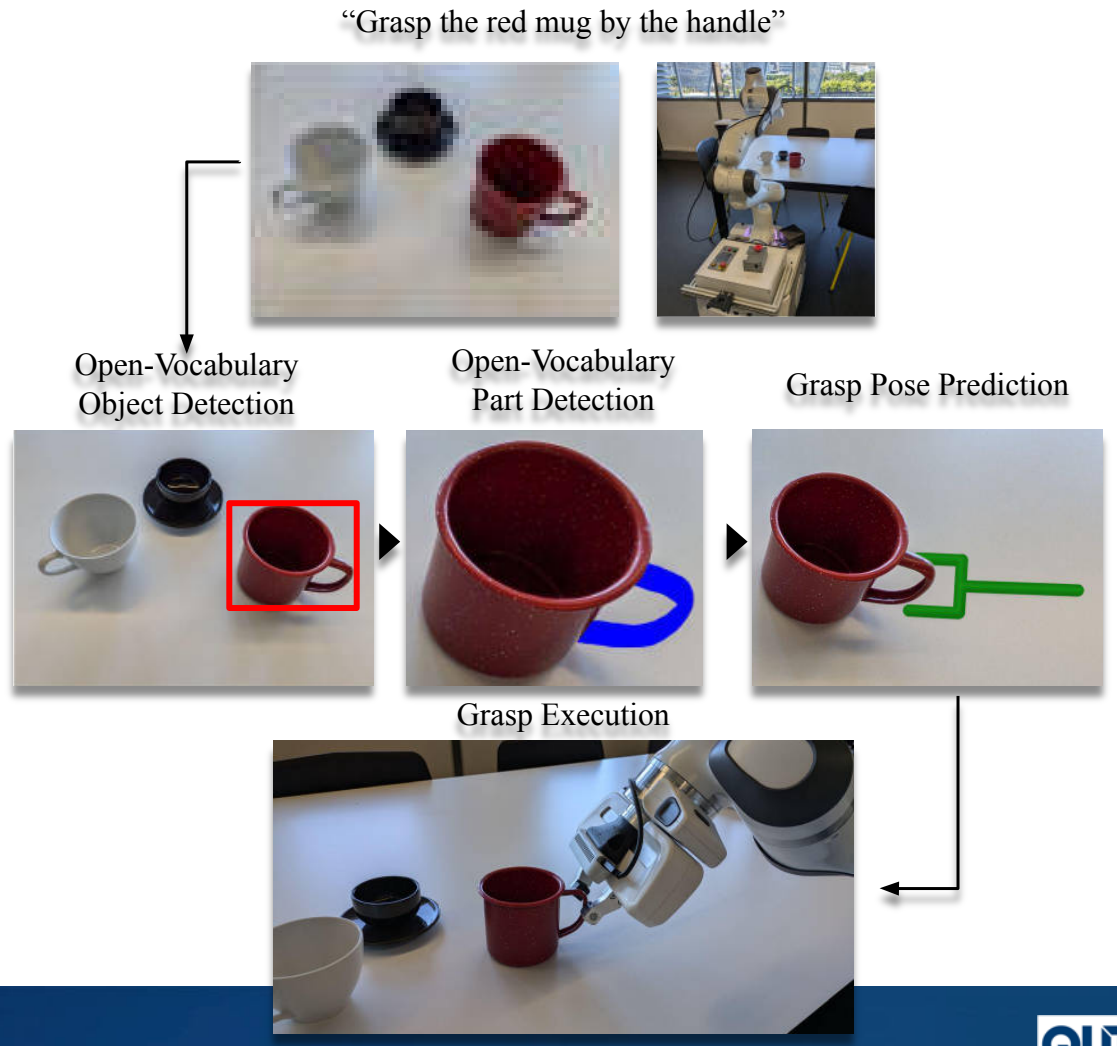


CRICOS No.00213J

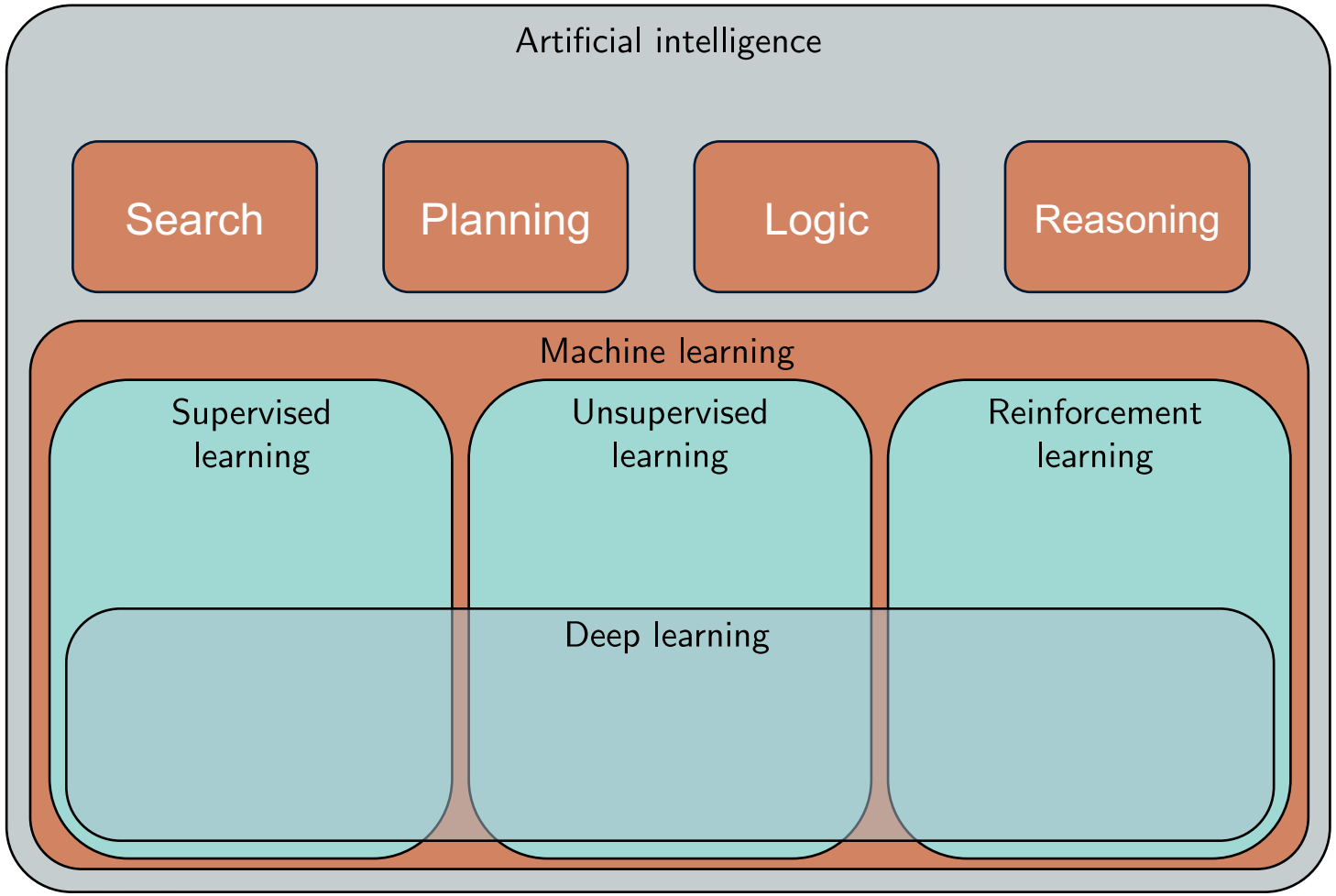

bottle|0.95
car|0.81
vase|0.935
vase|0.921
vase|0.864
vase|0.863
orange|0.88
book|0.84
clock|0.84
keyboard|0.96
mouse|0.95
bowl|0.97
teddy bear|0.98
cup|0.95
cup|0.96
spoon|0.97
ork|0.96
knife|0.76
cup|0.68
cup|0.92

Example – Grasping Objects

A robot can interact in a targeted way with the world by combining language understanding, semantic perception on multiple levels, and control.



Machine Learning vs AI vs Deep Learning



- **Artificial Intelligence:** Intelligence demonstrated by machines
- **Machine learning:** algorithms that to perform a specific task without using explicit instructions, relying on patterns and inference instead.
- **Machine Learning** algorithms build a mathematical model based on sample data, known as "training data", in order to make predictions or decisions without being explicitly programmed to perform the task

Don’t get lost in the details



CRICOS No.00213J

What do you remember from CAB420?

Have you used / played with / implemented any deep learning outside of CAB420?

Reminder: More Deep Learning Concepts



CRICOS No.00213J

Common Tasks



Start with the task

Common Tasks

Image Classification



“Warehouse”

- **Input:**
- **Output:**
- **Supervision:**
- **Loss (example):**
- **Useful Metric:**

Common Tasks

Image Classification



"Warehouse"

- **Input:**
  - RGB Image
- **Output:**
  - One class label for the whole image
- **Supervision:**
  - One class label per image
- **Loss (example):**
  - Cross-entropy
- **Useful Metric:**
  - Per-class accuracy

CRICOS No.00213J

Common Tasks

Semantic Segmentation

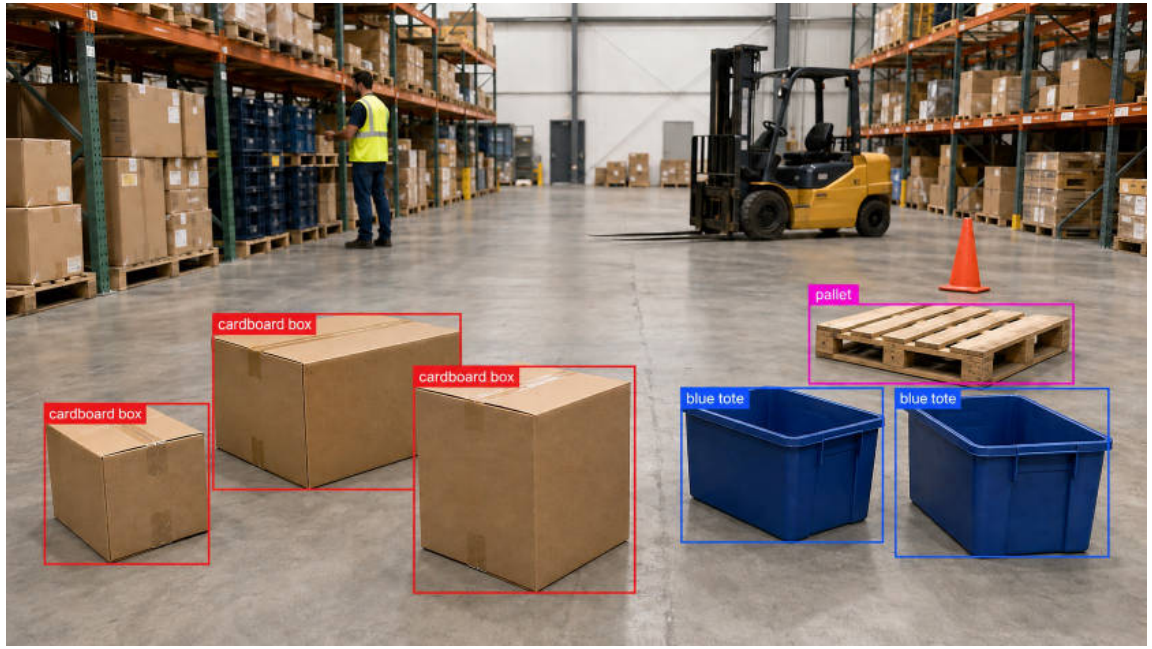


- **Input:**
  - RGB Image
- **Output:**
  - One semantic class per pixel, such as floor, robot, box or person
- **Supervision:**
  - Pixel-level semantic mask
- **Loss (example):**
  - Pixel-wise cross-entropy
- **Useful Metric:**
  - Mean Intersection-over-Union (IoU)

CRICOS No.00213J

Common Tasks

Object Detection



- **Input:**
  - RGB Image
- **Output:**
  - A set of bounding boxes, class labels and confidence scores, one per object
- **Supervision:**
  - Bounding box and class label for each object
- **Loss (example):**
  - Classification and bounding-box regression losses
- **Useful Metric:**
  - Average precision at an IoU threshold

CRICOS No.00213J

Common Tasks

Instance Segmentation

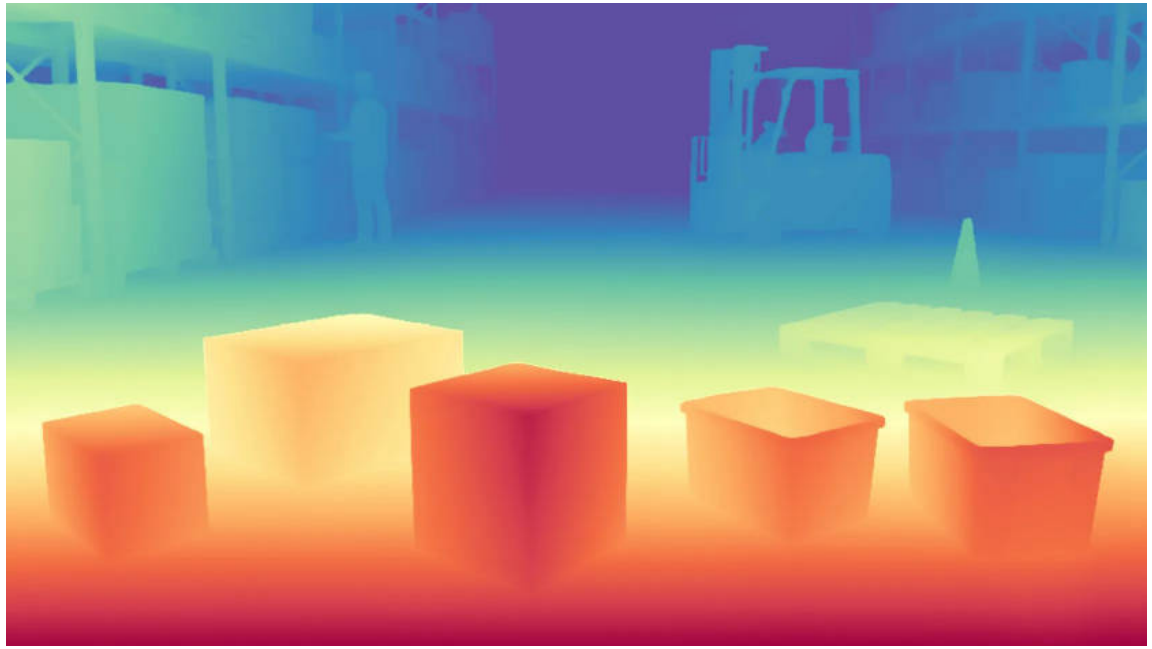


- **Input:**
  - RGB Image
- **Output:**
  - A separate pixel mask and class label for every object instance
- **Supervision:**
  - Individual mask and class label for each object
- **Loss (example):**
  - Classification, box-regression and mask losses
- **Useful Metric:**
  - Mask average precision

CRICOS No.00213J

Common Tasks

Depth Prediction

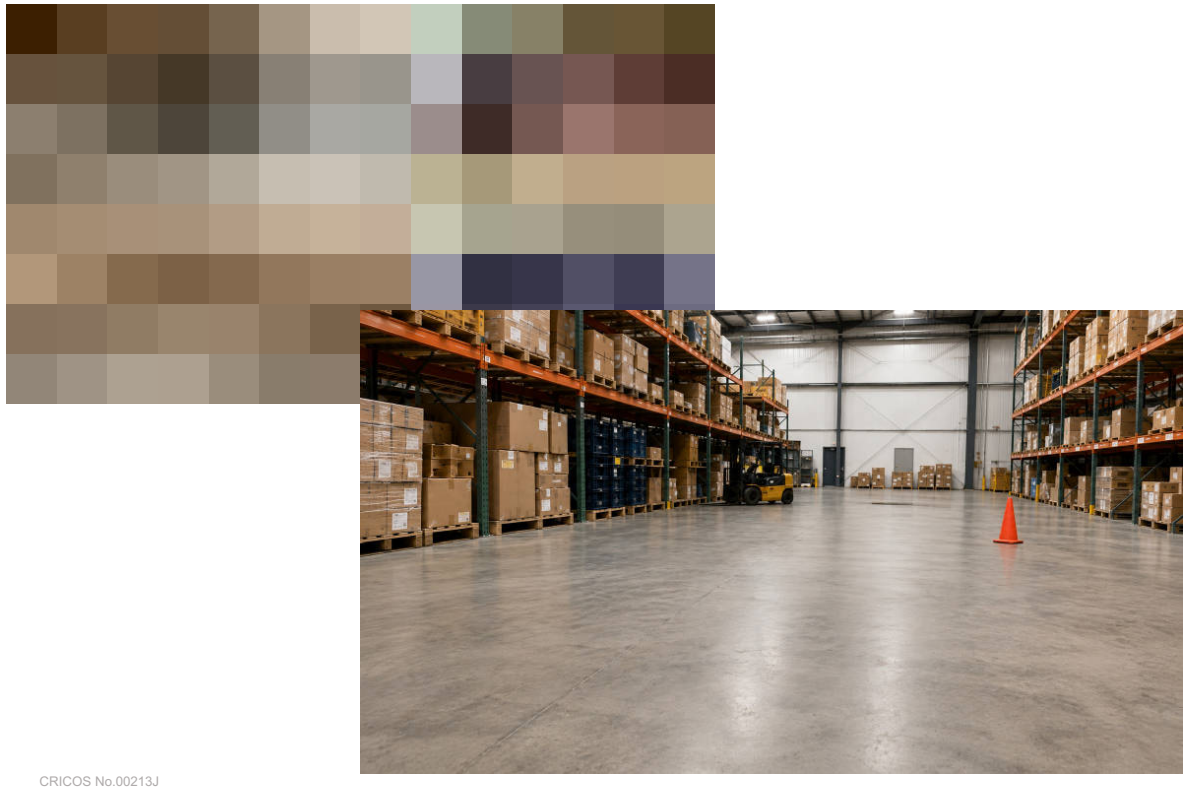


- **Input:**
  - RGB Image
- **Output:**
  - Estimated depth for every pixel
- **Supervision:**
  - Ground-truth depth map, e.g. from depth sensor or stereo system
- **Loss (example):**
  - L2 depth loss
- **Useful Metric:**
  - RMSE

CRICOS No.00213J

Common Tasks

Place Recognition / Image Retrieval



- **Input:**
  - A query image and a database of previously observed images
- **Output:**
  - A ranked list of database images or places, ordered by visual similarity
- **Supervision:**
  - Place identities, GPS/pose proximity, or positive and negative image pairs
- **Loss (example):**
  - Contrastive loss
- **Useful Metric:**
  - Recall@K, precision-recall

CRICOS No.00213J

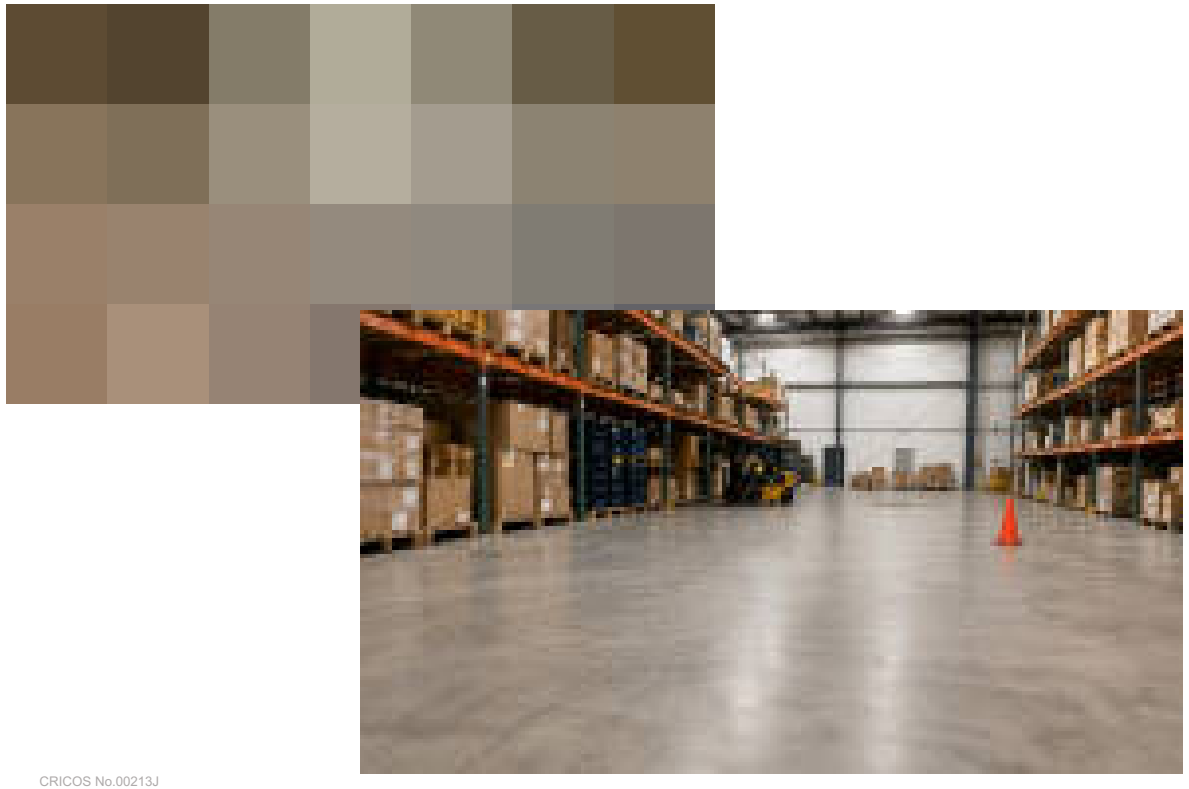
Example: Obstacle Warning



- **Input:**
- **Output:**
- **Supervision:**
- **Loss (example):**
- **Useful Metric:**

CRICOS No.00213J

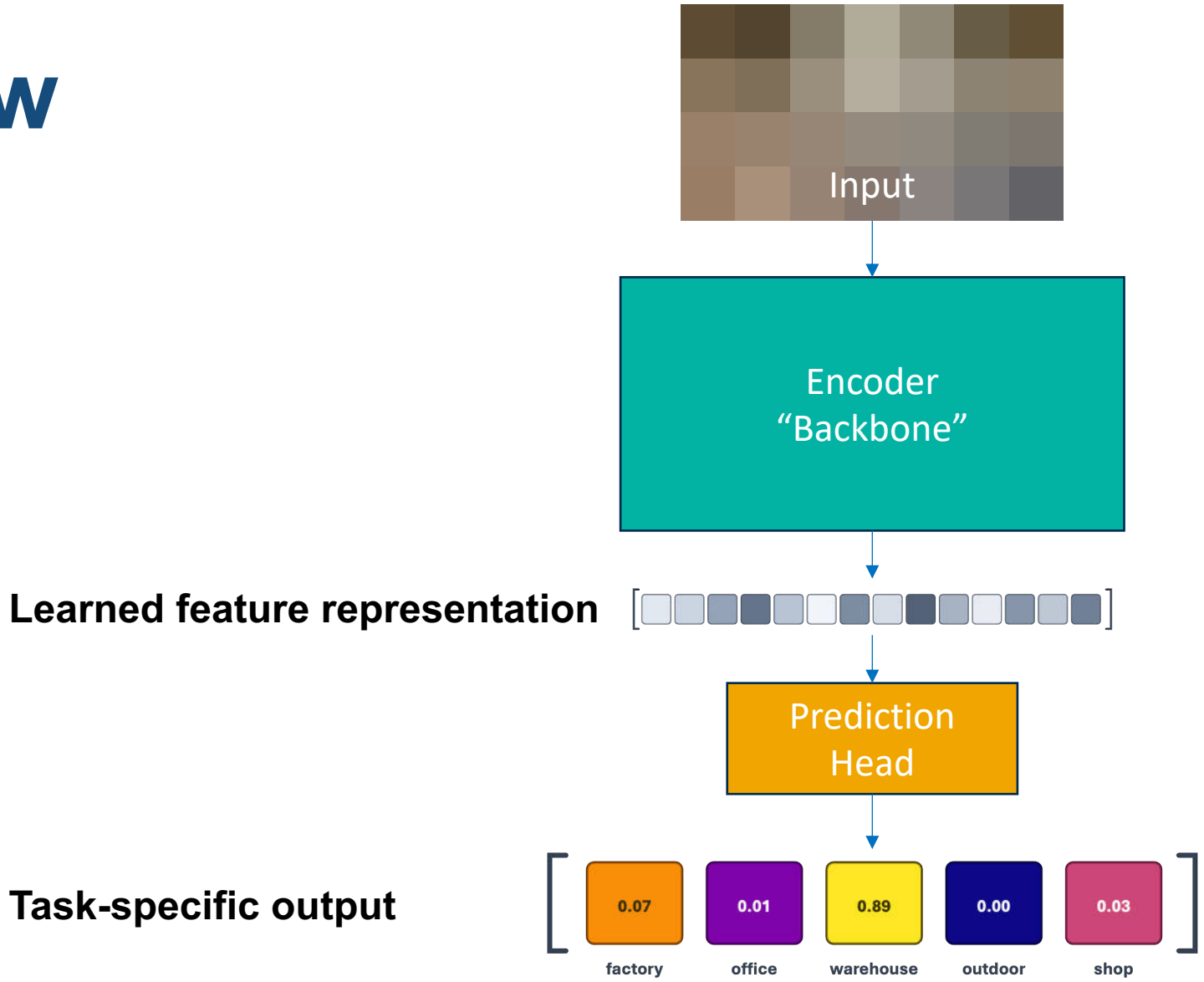
Example: Obstacle Warning



- **Input:**
  - RGB Image
- **Output:**
  - Probability that path is blocked
- **Supervision:**
  - Example images with blocked / clear labels
- **Loss (example):**
  - Binary cross-entropy
- **Useful Metric:**
  - Precision & recall
  - False-stop rate and missed-obstacle rate at chosen decision threshold

CRICOS No.00213J

A High-Level View

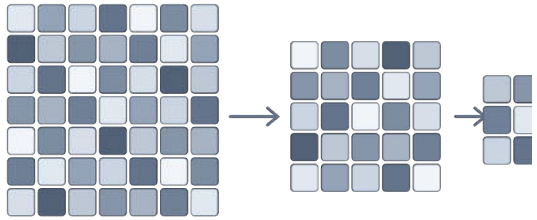


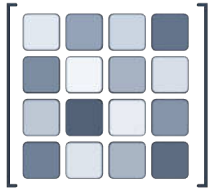
CRICOS No.00213J

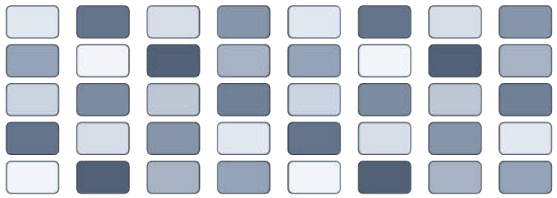
A High-Level View

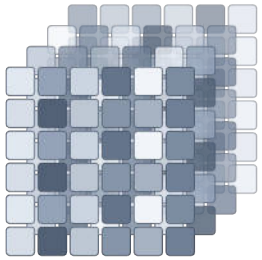
Learned feature representation / “embedding”

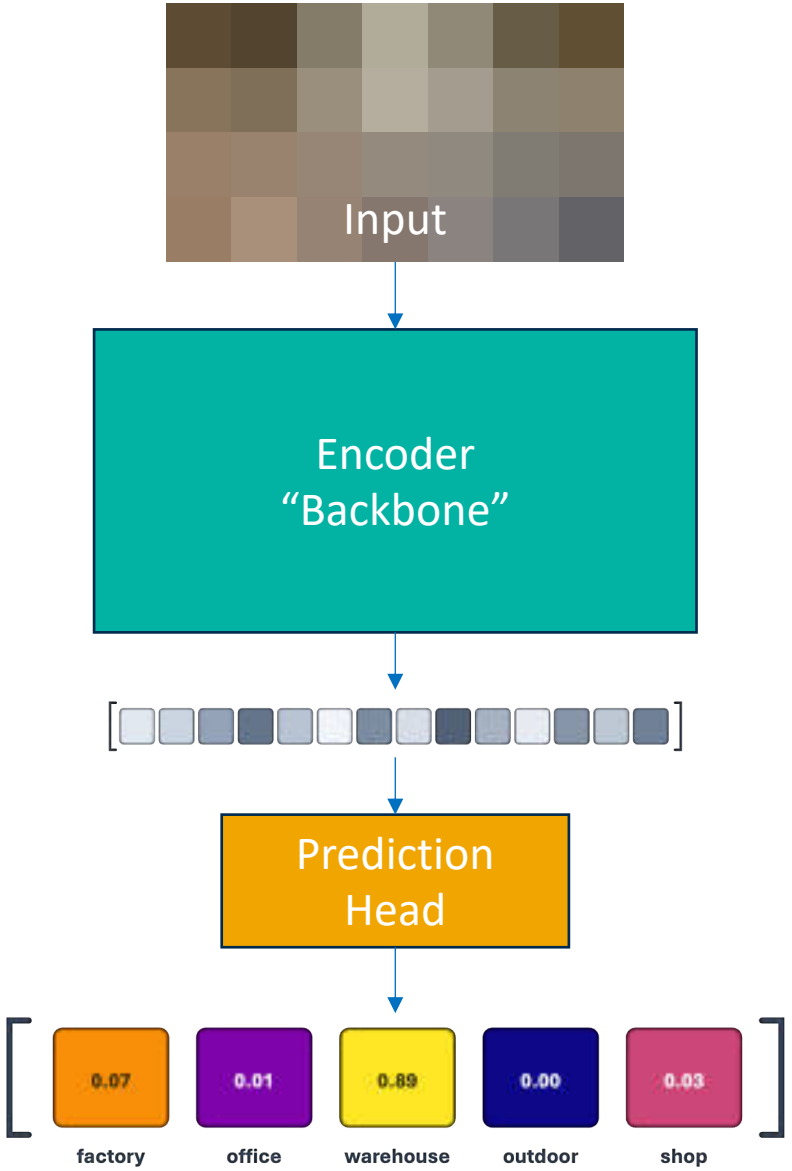
- The representation’s shape depends on the architecture and downstream task.
- It could be a vector, a spatial feature map, stacked feature map with multiple channels, multi-scale feature pyramid, patch token sequence …









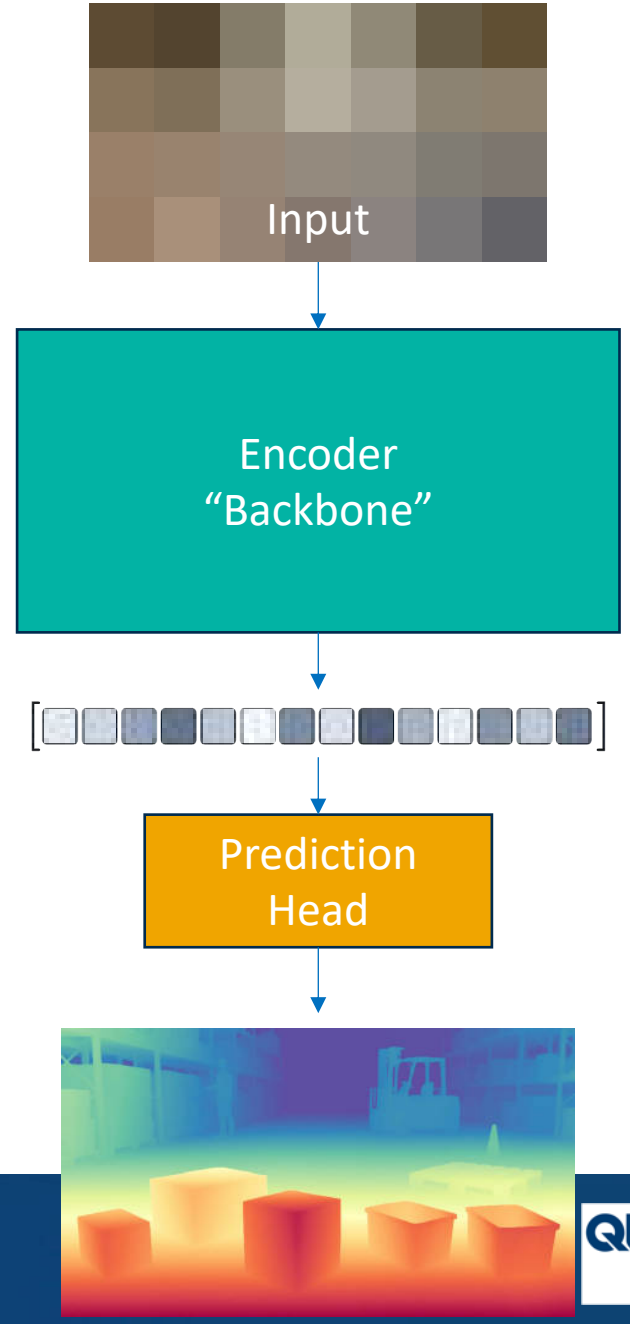


CRICOS No.00213J

A High-Level View

You will often re-use an existing, pre-trained backbone and only train the prediction head.

This is a form of **transfer learning**.



To train or not to train – or what to train

1	**Direct feature reuse**	Reuse embeddings without any task training. Fastest; fewest parameters. E.g. use features for nearest-neighbour search, clustering, retrieval.
2	**Linear probing / Train a task-head**	Train a simple prediction head on top of the backbone; keep the backbone “frozen”, i.e. unchanged.
3	**Partial fine-tuning**	Update selected later layers while freezing early layers.
4	**Full fine-tuning**	Update the entire model. More compute and data required.
5	**From scratch**	Learn all parameters without pretrained weights. Maximum data demand.

CRICOS No.00213J

To train or not to train – or what to train

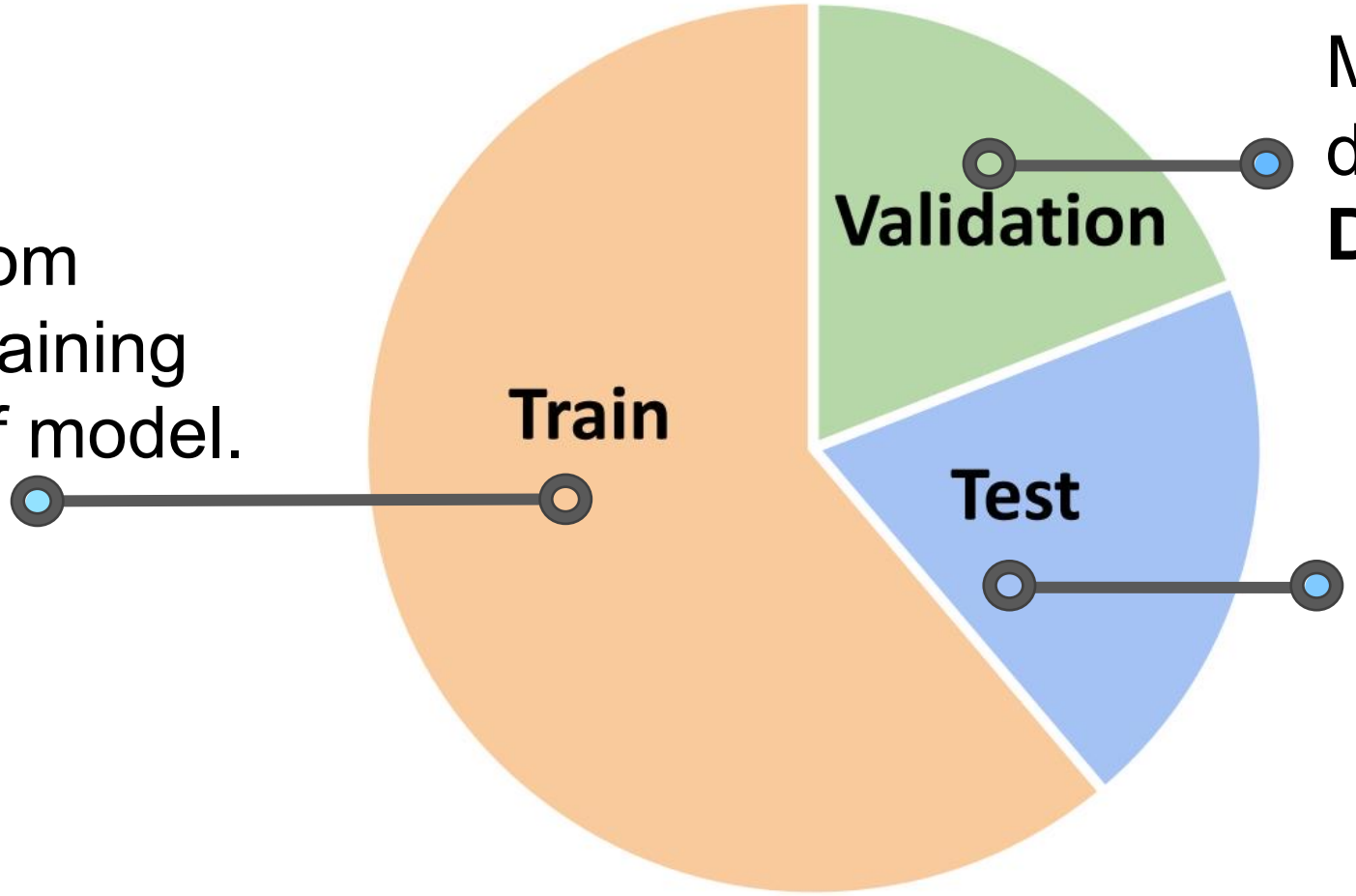
Approach	Use it when	Main limitation
Direct feature reuse	You have few or no labels; the task is retrieval, similarity, clustering or place recognition; the pretrained embedding already separates relevant examples	Cannot reshape the representation for your task
Linear probing / Train a task head	You have labelled data but want to keep the pretrained backbone frozen; train a linear probe, MLP, detection head or decoder on top of its features	A more capable head may help, but the backbone cannot adapt
Partial fine-tuning	The target domain differs from pretraining; a linear probe is insufficient; the labelled dataset is moderate rather than large	Requires decisions about which layers to unfreeze
Full fine-tuning	You have enough representative labelled data; the target task or domain differs substantially; maximum task performance matters	Higher compute cost and greater overfitting risk
Training from scratch	No suitable pretrained model exists; the input modality or architecture is unusual; or you have a very large task-specific dataset	Usually requires the most data, compute and experimentation

- In general fewer labels and less compute favour frozen models. More task data and a larger domain gap favour fine-tuning.
- **More fine-tuning is not automatically better.** With a small dataset, updating the whole network can destroy useful pretrained features or overfit.
- **Linear probing is diagnostically useful.** If it performs well, the pretrained representation already contains the information needed for the task. If full fine-tuning improves substantially, the representation needed adaptation.

CRICOS No. 00213J

Dataset splits: Train, Val, Test

Learn from during training phase of model.



Monitor performance during training.
Do not learn from it.

Estimate model's real-world performance **after** training.

CRICOS No.00213J

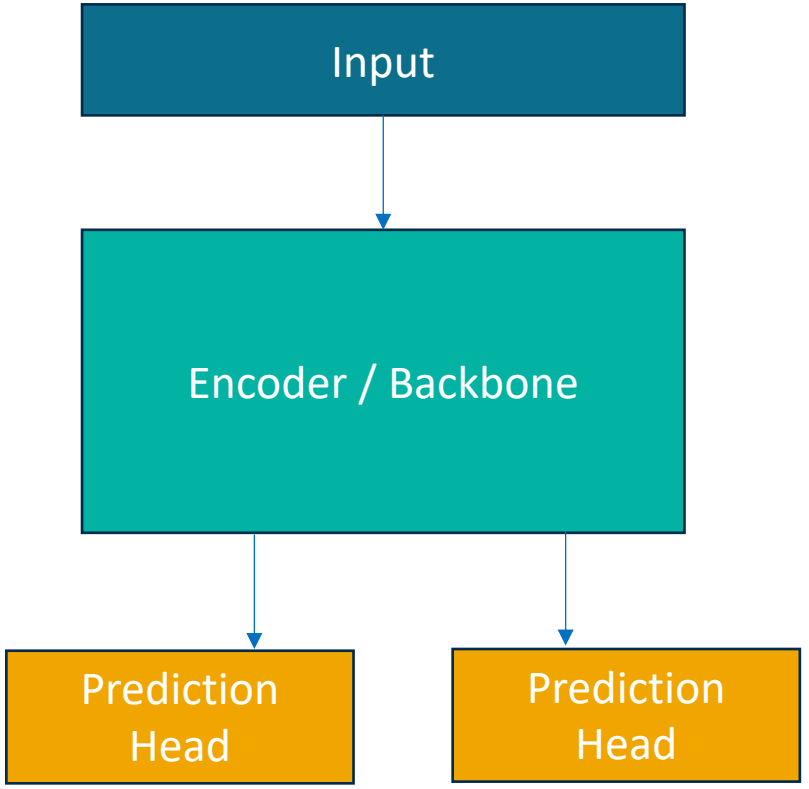
A Shared Vocabulary

Data	Examples drawn from an assumed operating distribution.
Backbone	Converts pixels into reusable feature representations.
Task head	Maps features to task-specific outputs (replaceable).
Loss	Training signal used to update model parameters.
Metric	External measure used to compare deployed behaviour — not the loss.

A High-Level View

- **Start with the task, not the architecture:**
  - What is the input?
  - What are desired outputs?
  - What supervision signals are available?
  - What loss rewards desired behaviour?

- Many architectures are possible
- A common backbone can be shared or re-used for multiple tasks or outputs



CRICOS No.00213J

Where does the training signal come from?

Supervised Learning

- **Labels** created for a particular task
- Examples: image classes, bounding boxes, segmentation masks and depths
- Directly teaches the model the required output
- Labels often expensive to collect at scale

Self-Supervised Learning

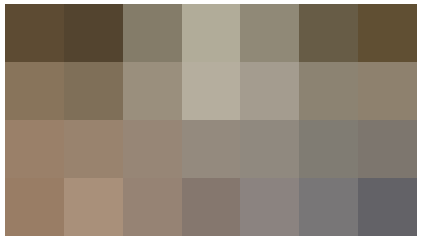
- Training targets are constructed from **unlabelled** images
- Goal: create rich representations that can be adapted easily for many tasks
- The model learns by comparing views, predicting representations or reconstructing missing content
- Usually followed by fine-tuning or adaptation to a downstream task

CRICOS No.00213J

Self-Supervised Learning (Triplet Loss)

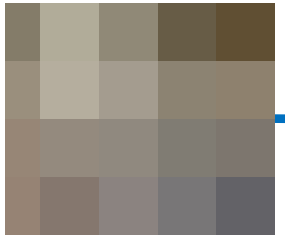


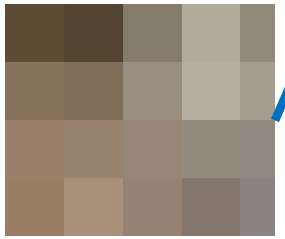
Choose two images



Random crops







Encoder
“Backbone”



Loss: Make these two embeddings **less** similar



Loss: Make these two embeddings **more** similar



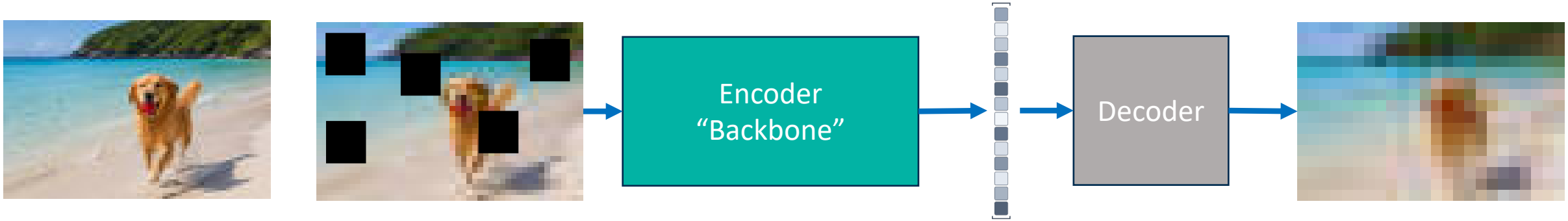
CRICOS No.00213J

Self-Supervised Learning (Masked Autoencoder)

Original Image

Randomly mask out parts of the image

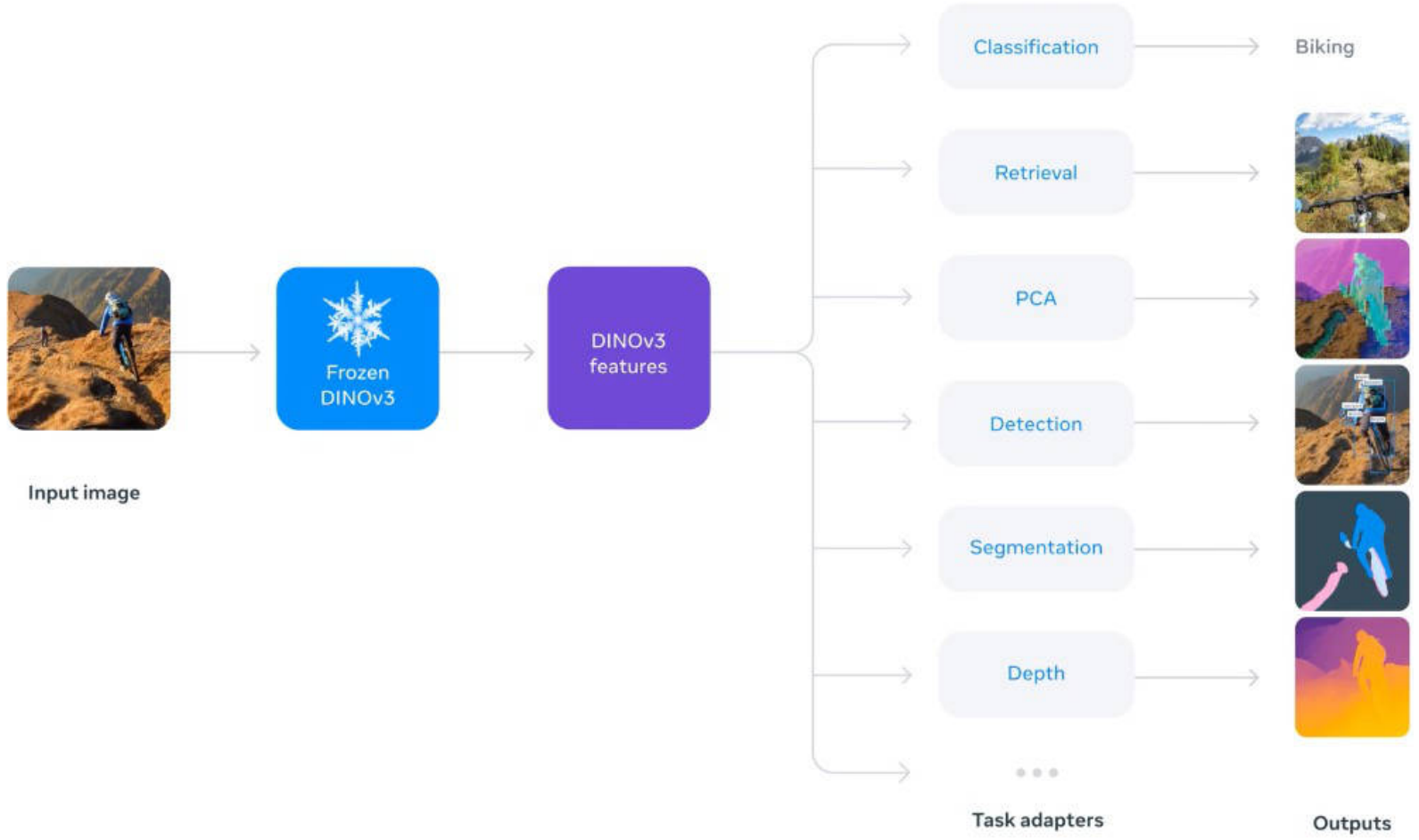
Reconstructed Image



Loss: Regenerate the original image

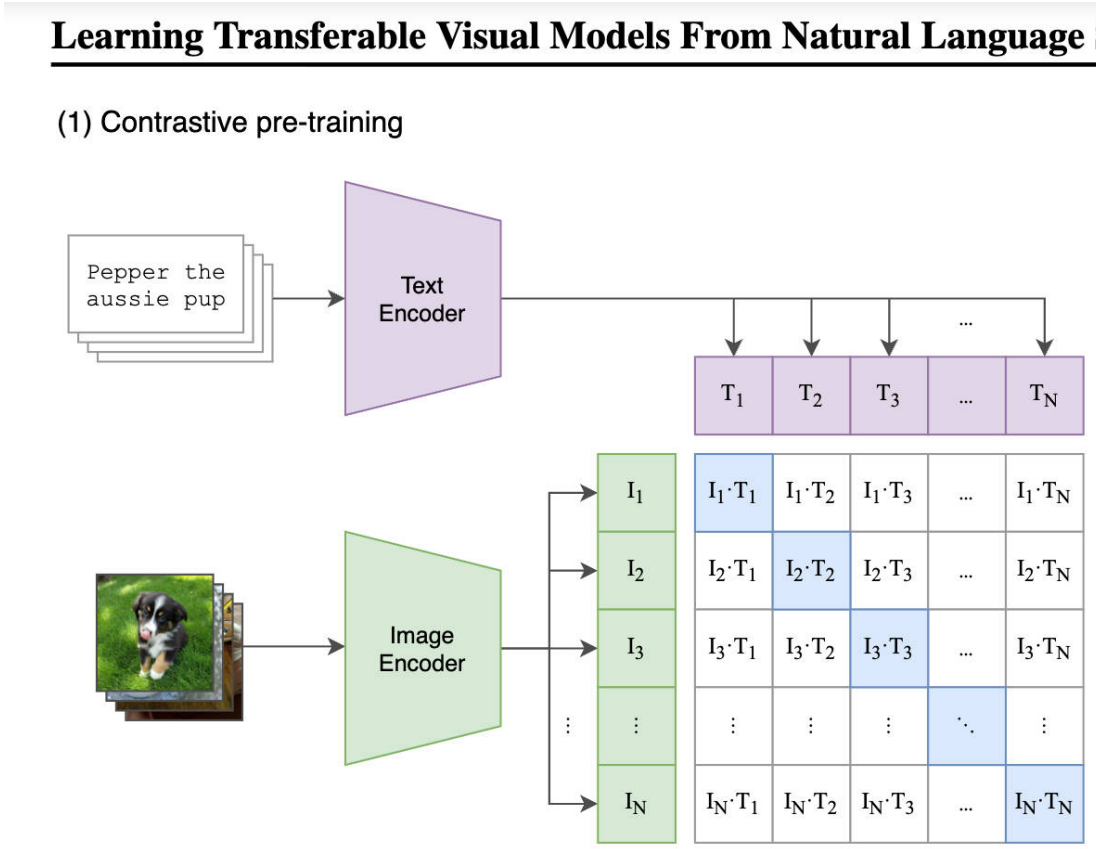
Self-Supervised Learning (DINOv3)

- Student-Teacher Distillation
- Students attempts to emulate teacher networks output
- Both get to see multiple views of the same images
- More in the tutorial!

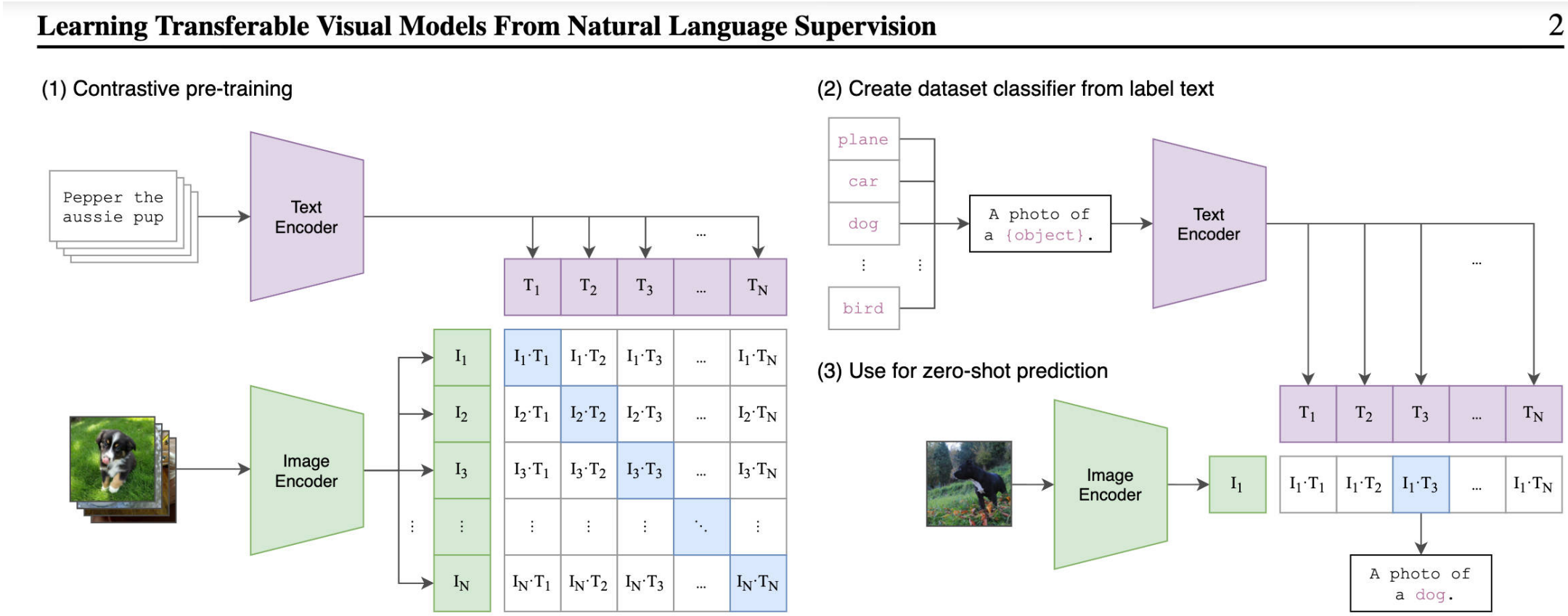


CRICOS No.00213J

Self-Supervised Learning (CLIP – Constrastive Language-Image Pre-training)

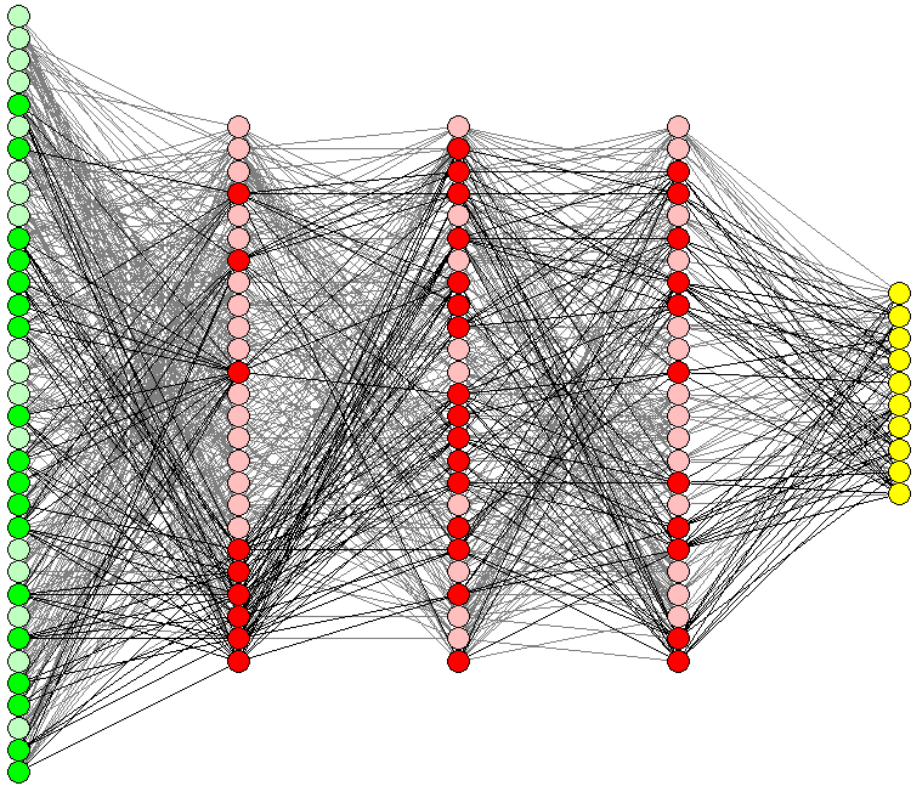


Self-Supervised Learning (CLIP – Constrastive Language-Image Pre-training)



Reminder: Most Common Architectures

Reminder: Fully Connected Layers



CRICOS No.00213J



Shape: (32,32,3)

Shape: (3072,1)

x

W^T

y

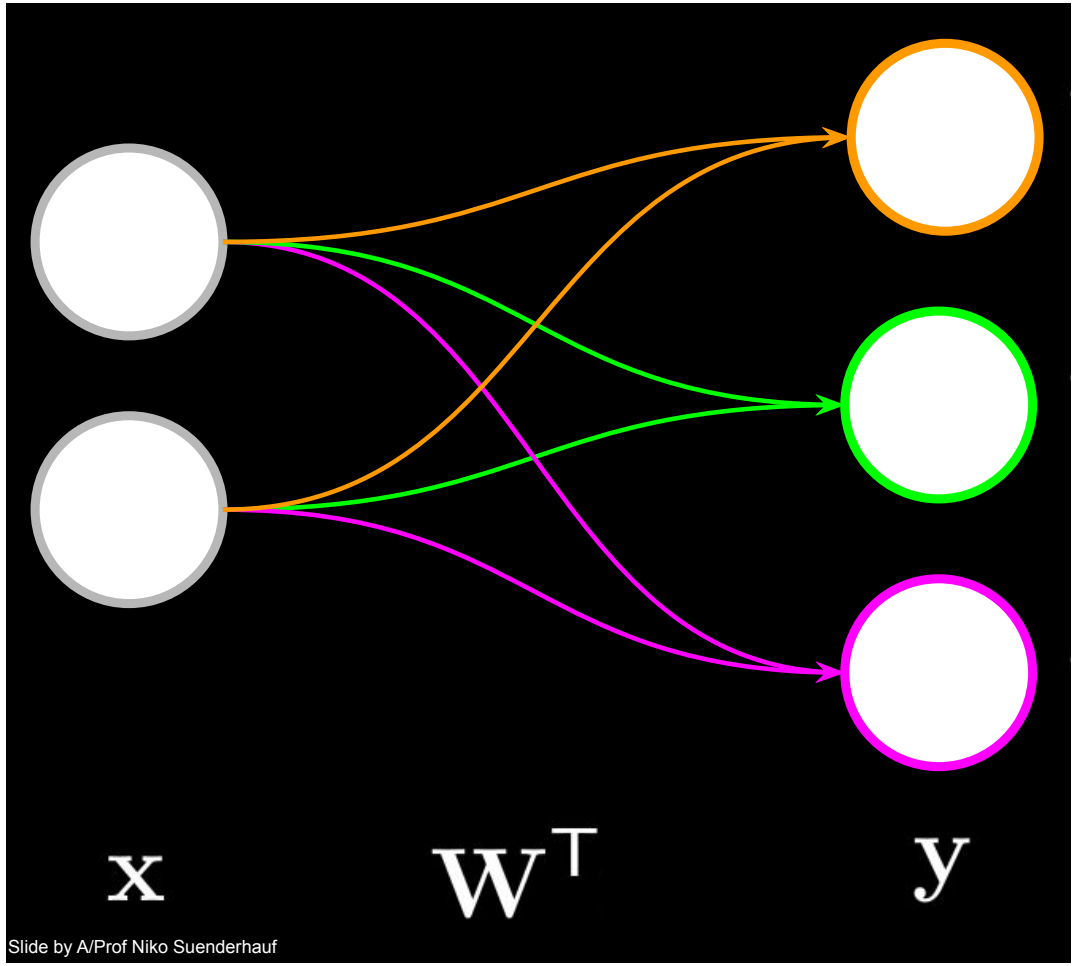
Airplane
Car
Bird
Cat
Deer
Dog
Frog
Horse
Ship
Truck

Slide by A/Prof Niko Suenderhauf

Slide by A/Prof Niko Suenderhauf

Architectures of Neural Networks

- What does a single-layer, fully-connected neural network do?

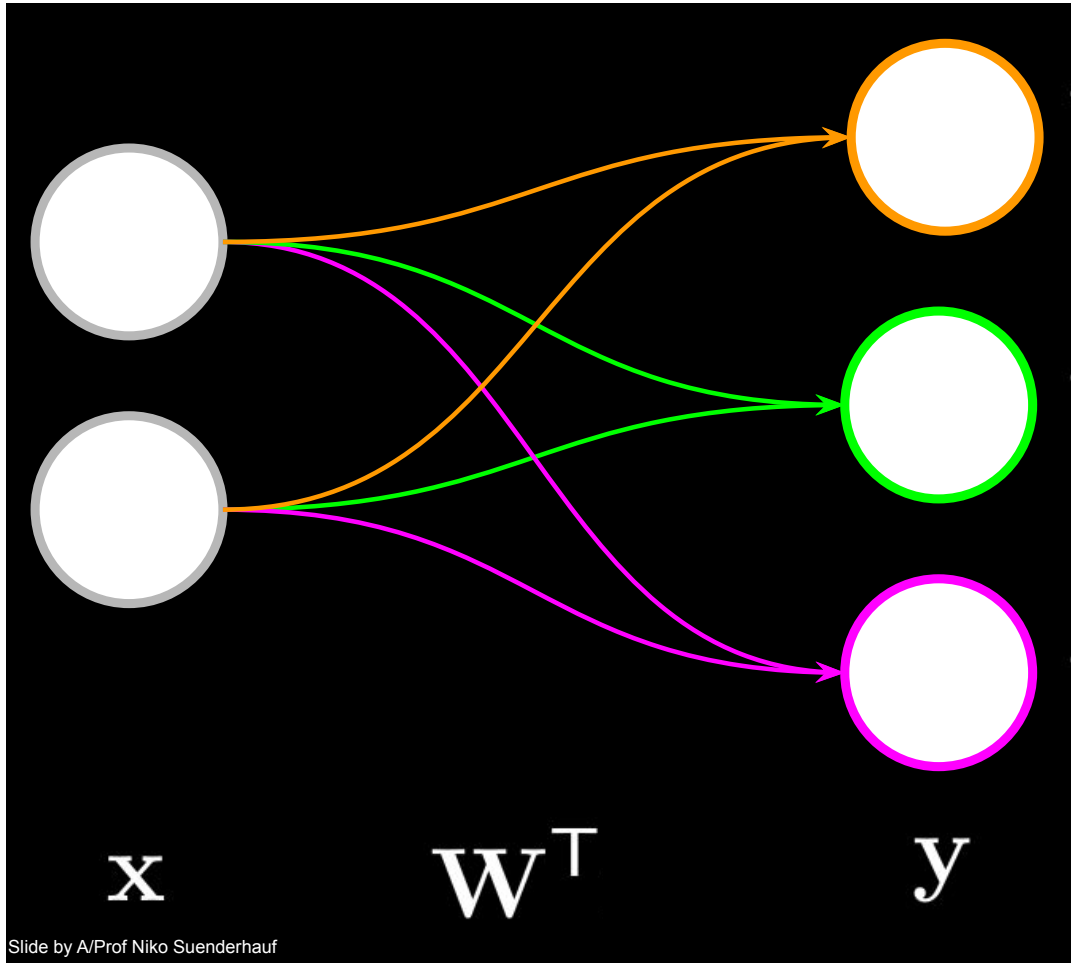


Slide by A/Prof Niko Suenderhauf

Architectures of Neural Networks

- What does a single-layer, fully-connected neural network do?

$$\mathbf{W} = \begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,2} \\ w_{1,0} & w_{1,1} & w_{1,2} \end{pmatrix}$$



Slide by A/Prof Niko Suenderhauf

Architectures of Neural Networks

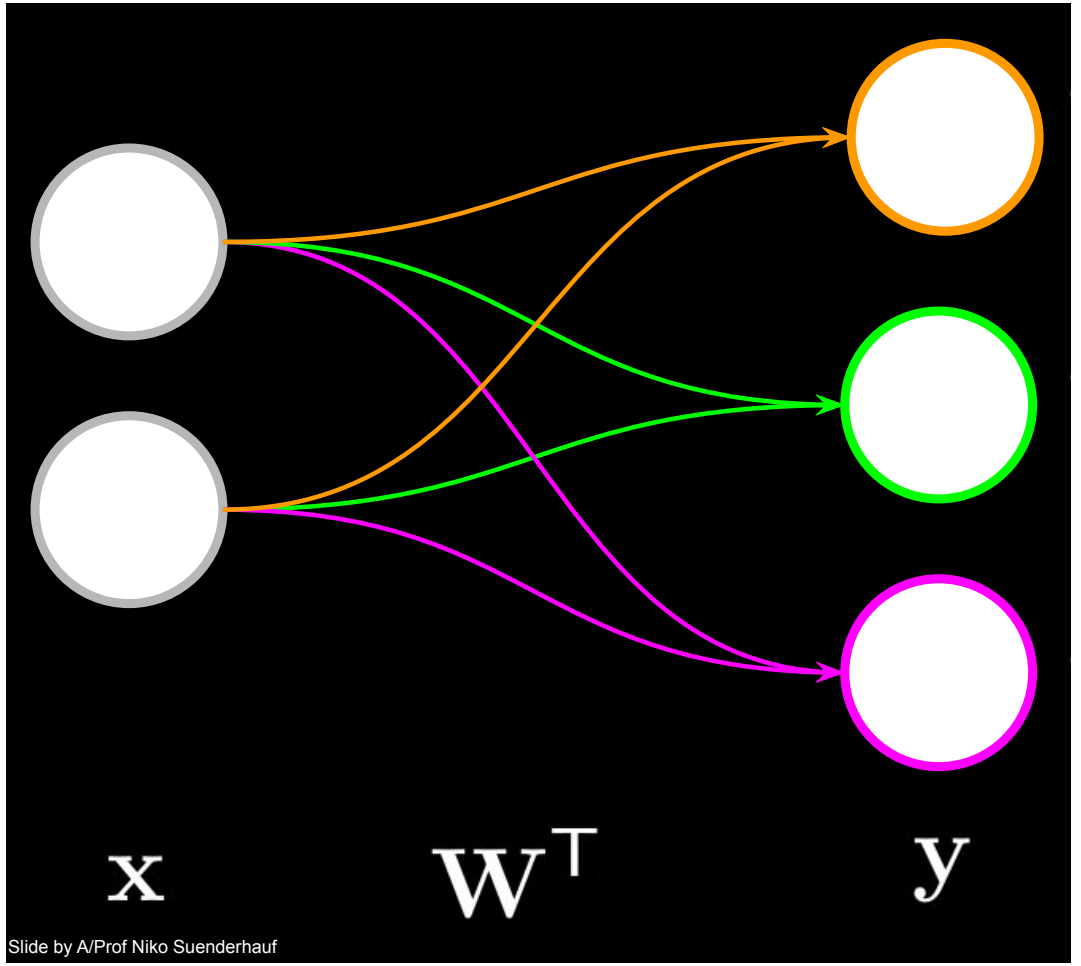
- What does a single-layer, fully-connected neural network do?

$$\mathbf{W} = \begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,2} \\ w_{1,0} & w_{1,1} & w_{1,2} \end{pmatrix}$$

$$y_0 = a(w_{0,0} \cdot x_0 + w_{1,0} \cdot x_1 + b)$$

activation function
(e.g. sigmoid, ReLU)

bias term



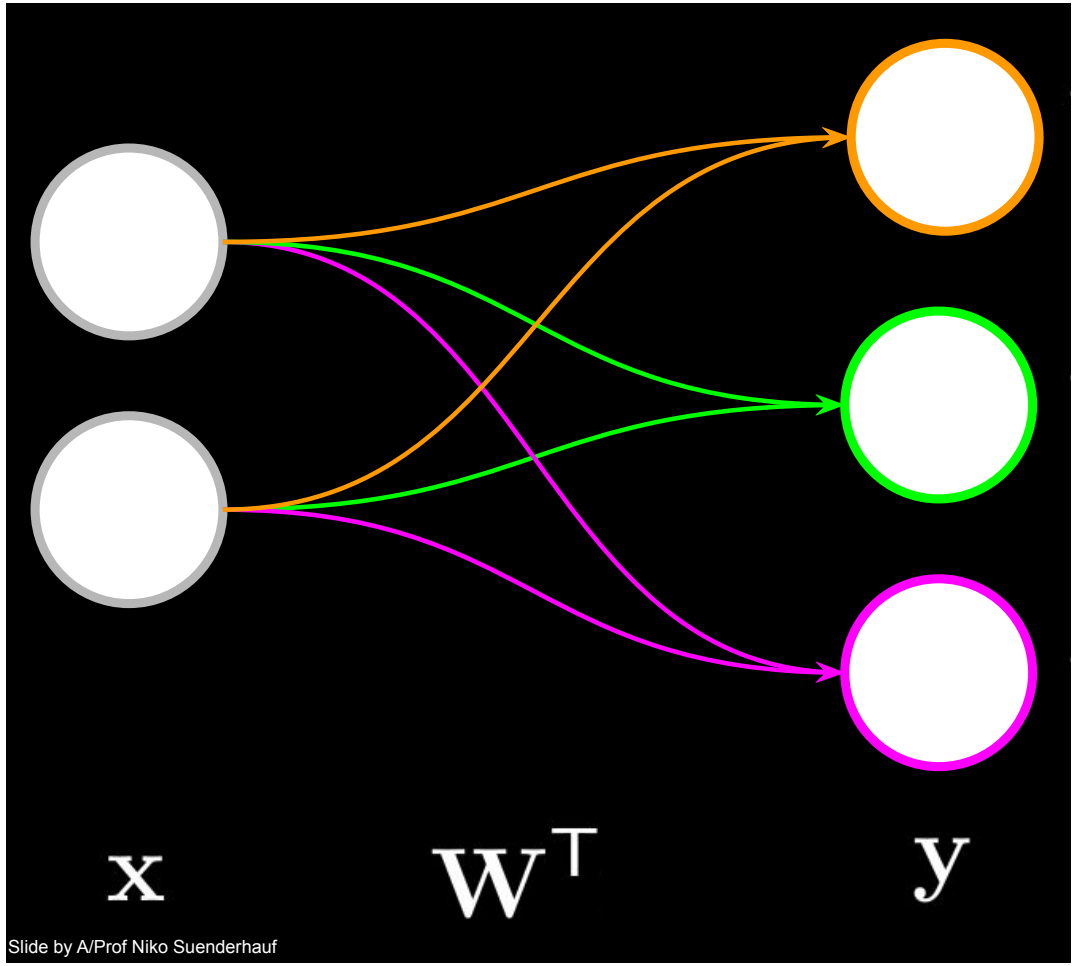
Slide by A/Prof Niko Suenderhauf

Architectures of Neural Networks

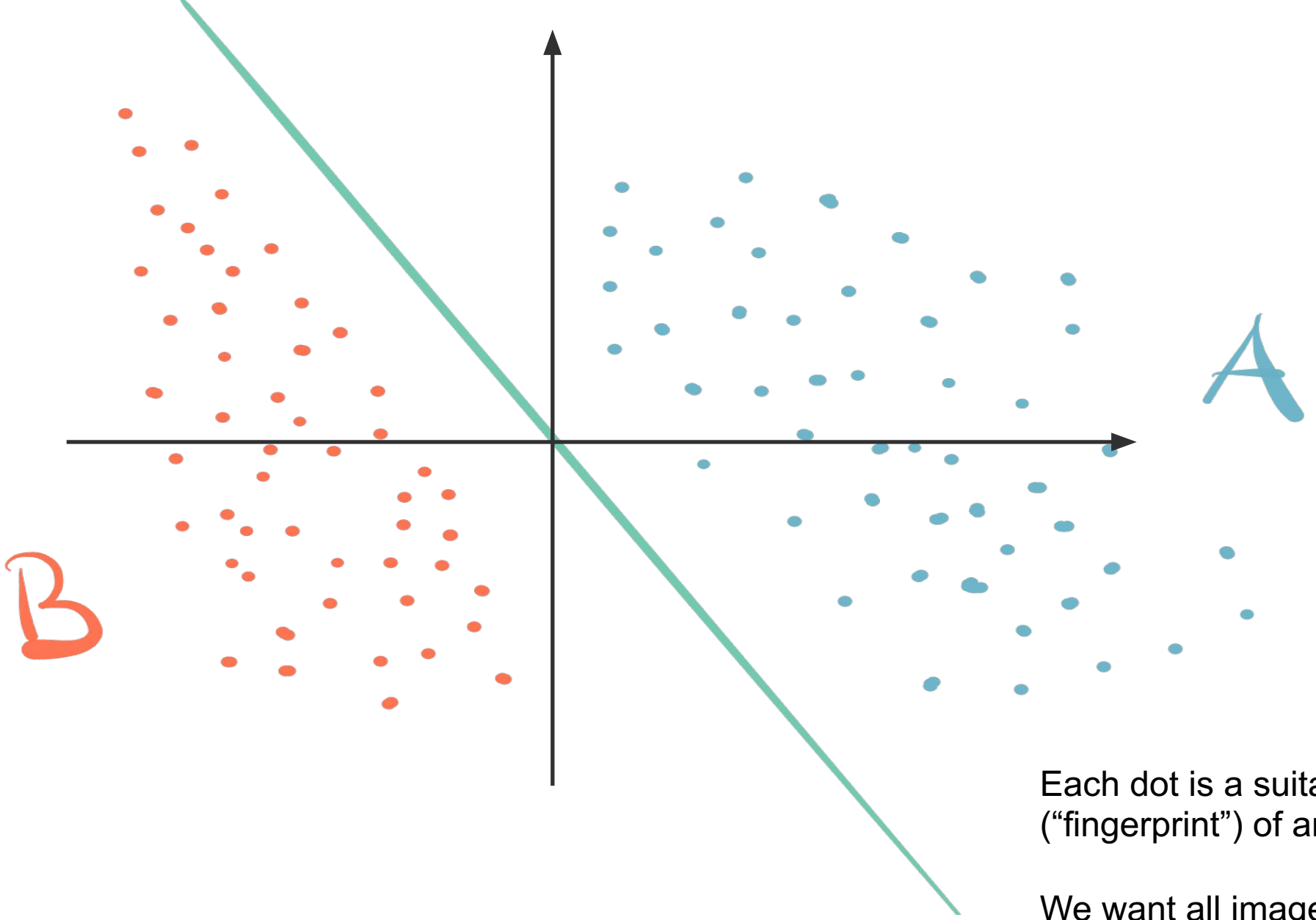
- What does a single-layer, fully-connected neural network do?

$$\mathbf{W} = \begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,2} \\ w_{1,0} & w_{1,1} & w_{1,2} \end{pmatrix}$$

$$\boldsymbol{y} = a\left(\boldsymbol{W}^T \boldsymbol{x} + \boldsymbol{b}\right)$$



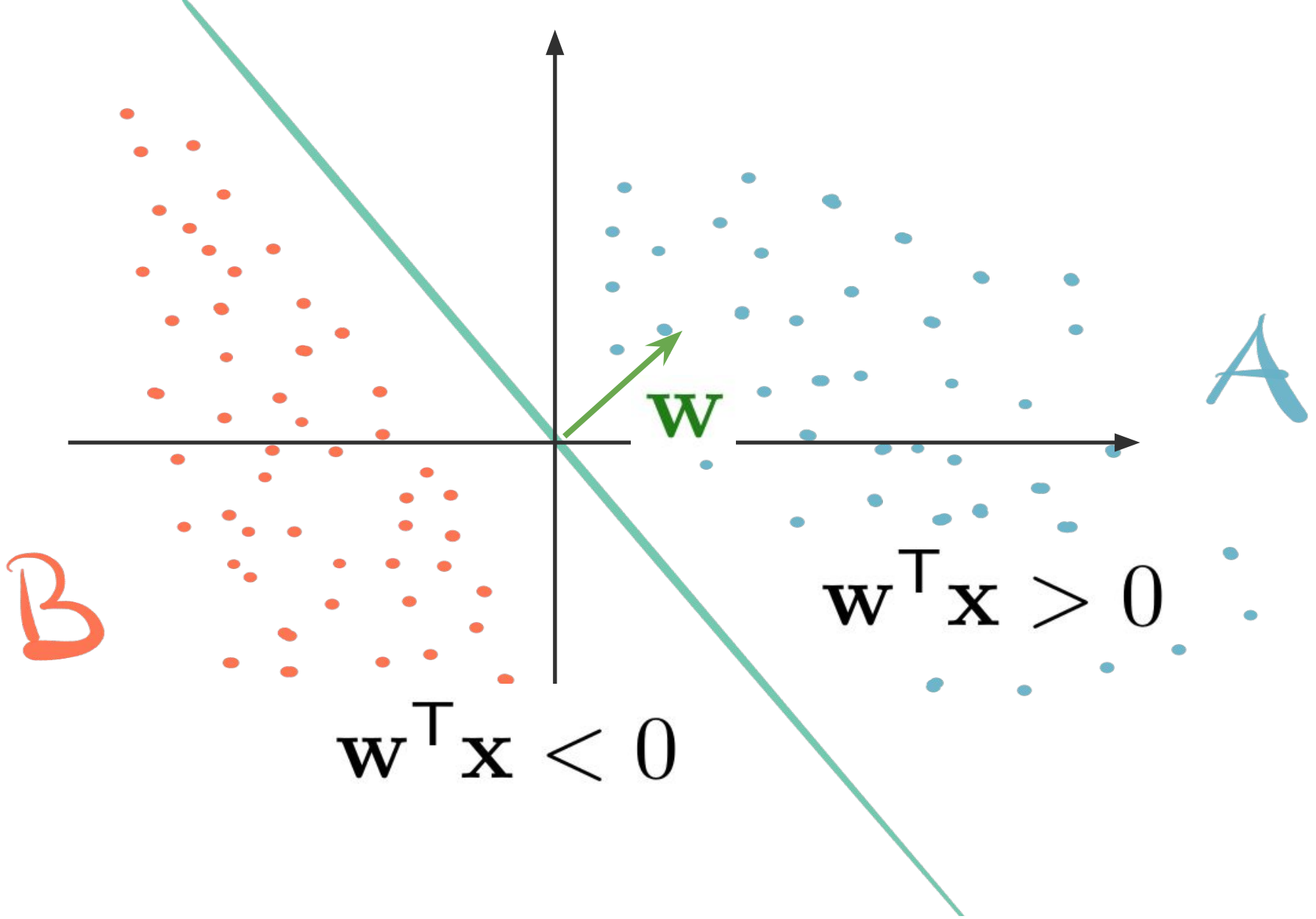
Slide by A/Prof Niko Suenderhauf



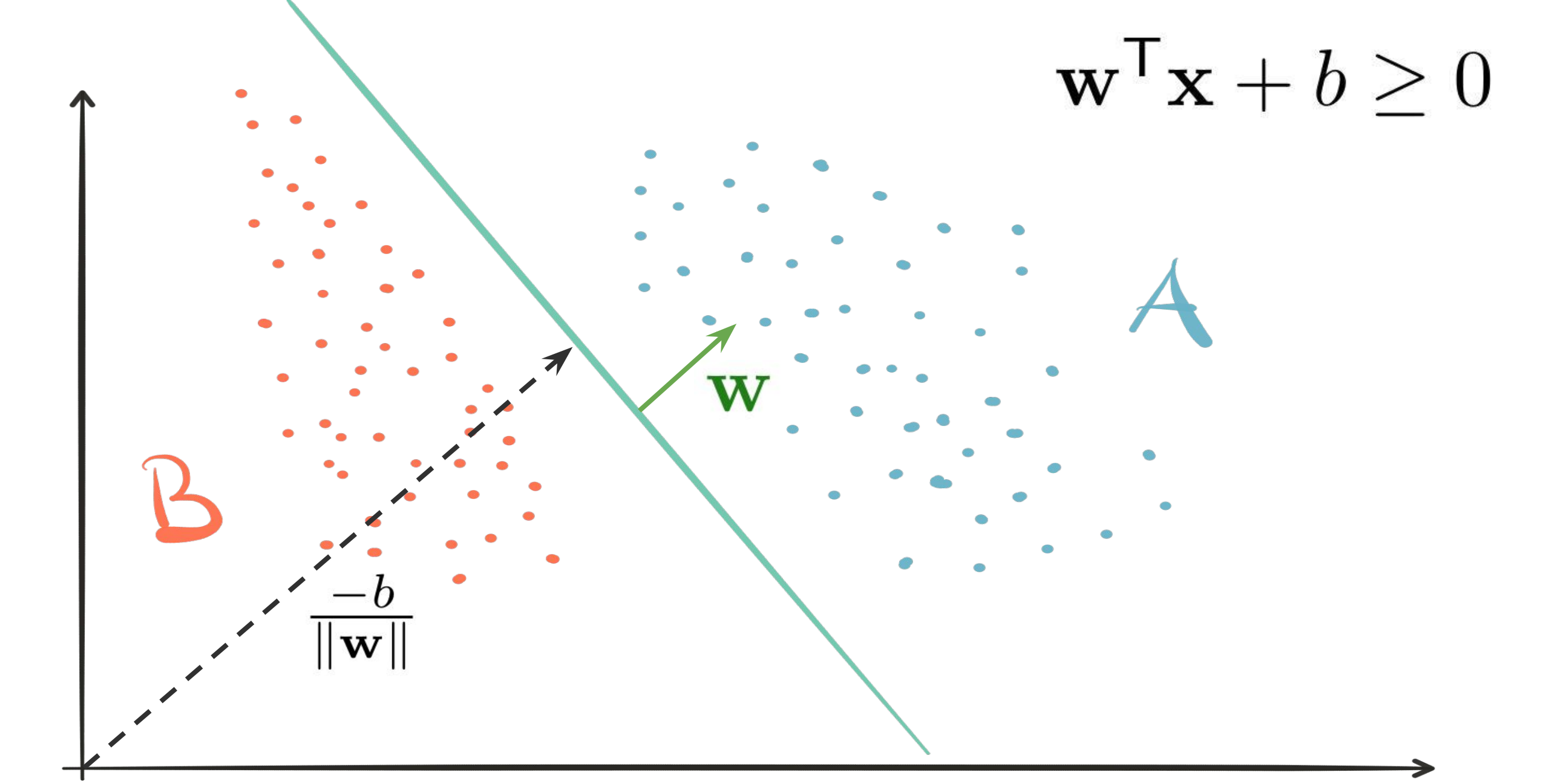
Each dot is a suitable representation (“fingerprint”) of an image.

We want all images of class A to be separated from all images of class B.

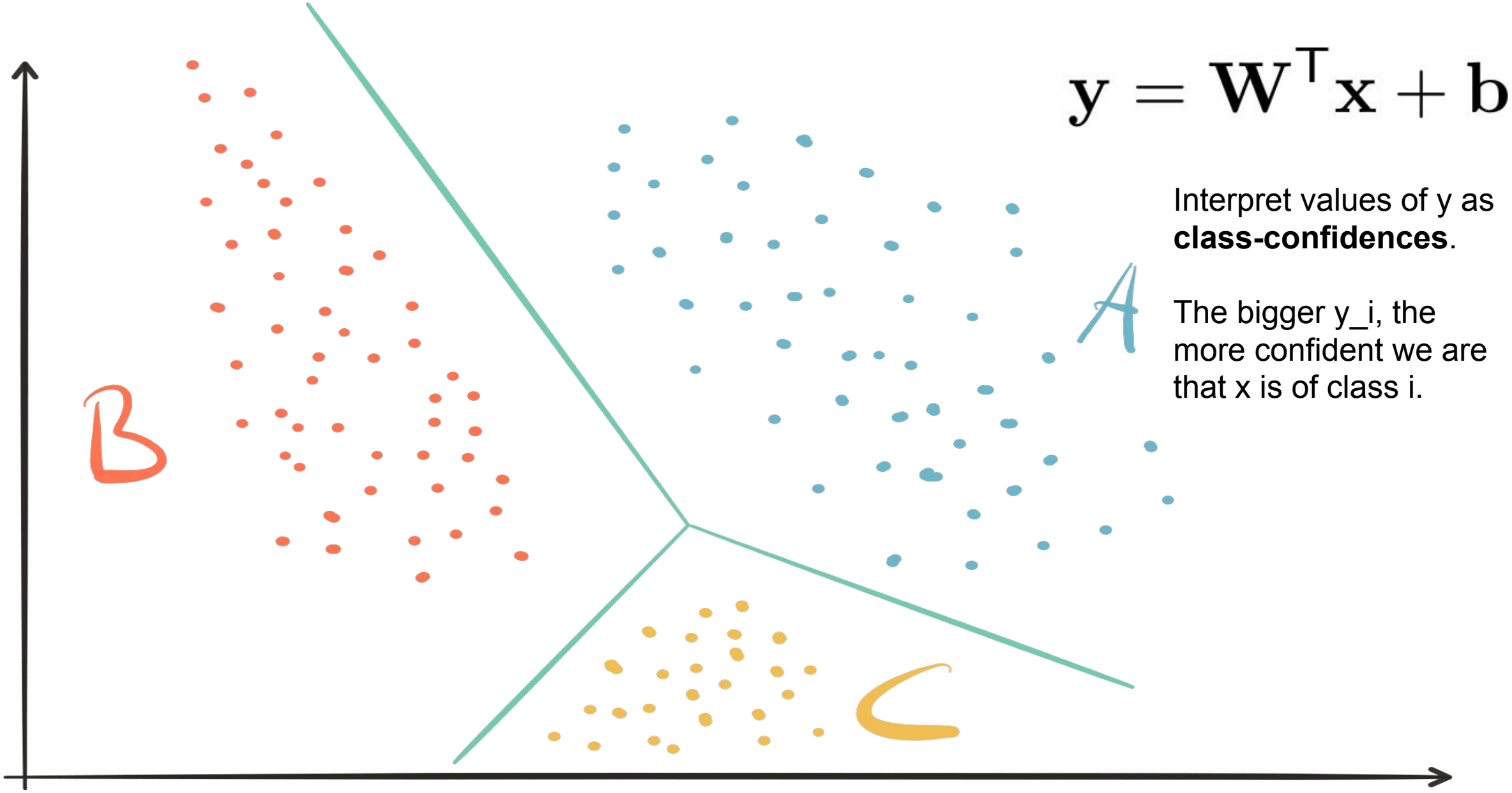
Slide by A/Prof Niko Suenderhauf



Slide by A/Prof Niko Suenderhauf



Slide by A/Prof Niko Suenderhauf



$$\mathbf{y} = \mathbf{W}^{\mathsf{T}}\mathbf{x} + \mathbf{b}$$

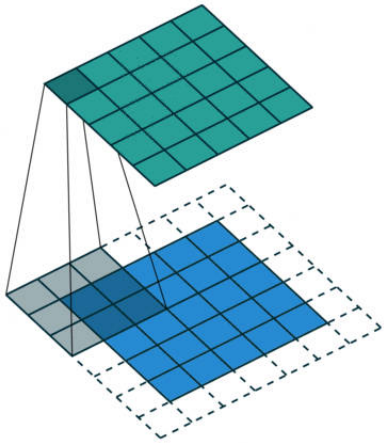
Interpret values of y as **class-confidences**.

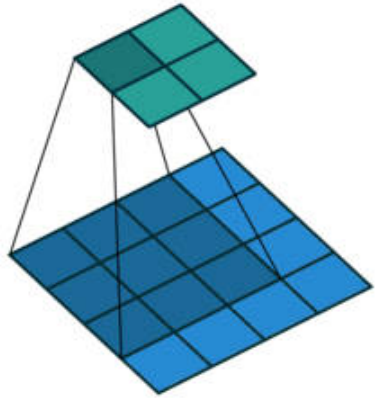
The bigger y_i, the more confident we are that x is of class i.

Slide by A/Prof Niko Suenderhauf

Reminder: Convolutional Layer

- A convolution applies the same local filter across all image locations.
- Weight sharing reduces parameters and builds in a translation-related inductive bias.
- Local layers compose into increasingly large receptive fields.
- Translation **equivariance**: a shifted input tends to produce a shifted feature response. This is NOT the same as invariance.



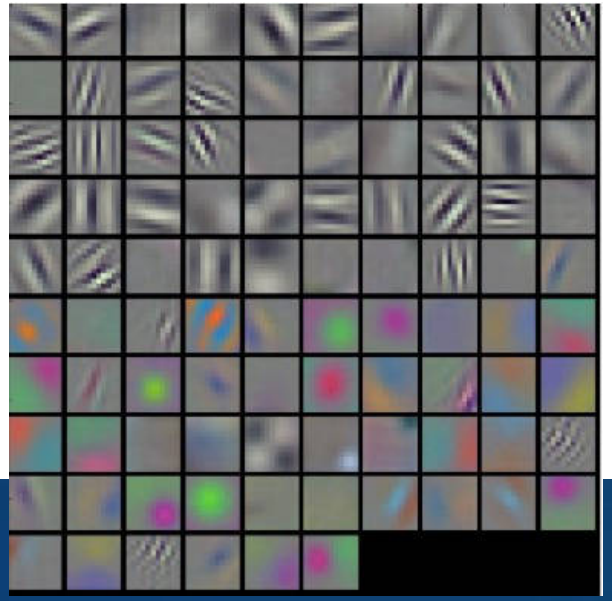


Equivariance

Shift input → feature map shifts.
Output location changes; pattern preserved.

Invariance

Shift input → output unchanged.
CNNs do NOT automatically provide this.



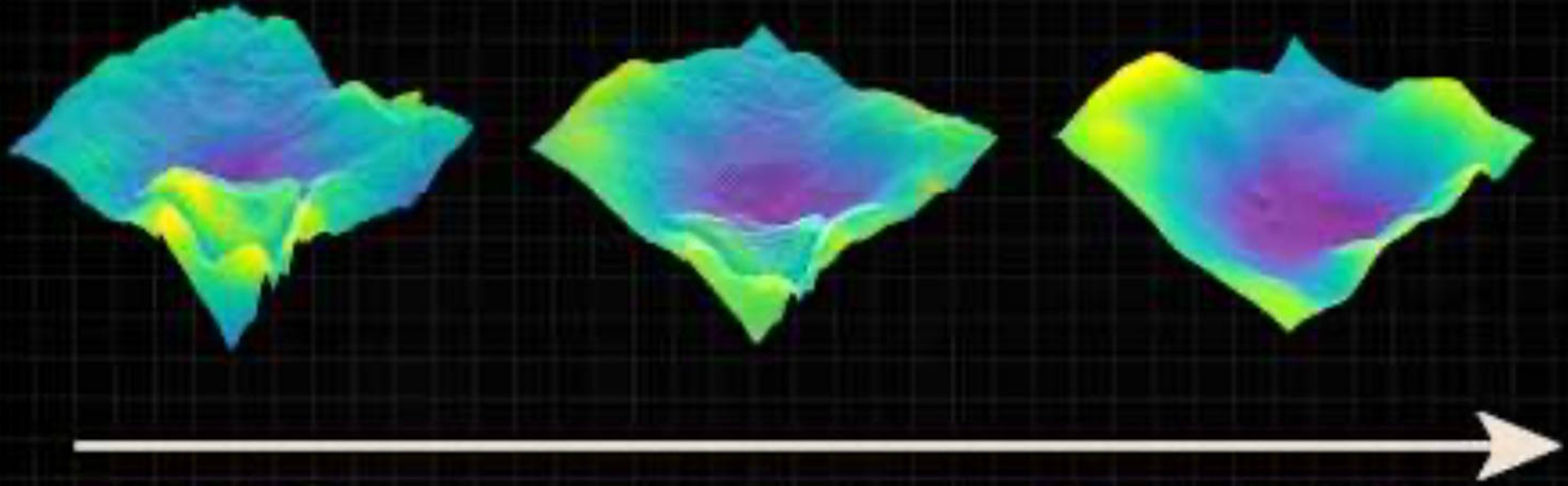
Reminder: Convolutional Layer

The following video is from Welch Labs (https://www.welchlabs.com/)
I highly encourage you to check out their educational videos on deep learning!

Identify:

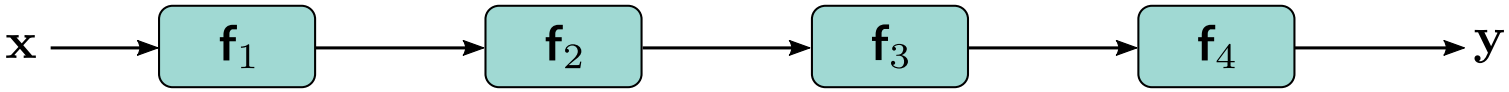
1. What is the representation of the image in this example?
2. What is the task-dependent prediction head?
3. What is the output?

CRICOS No.00213J

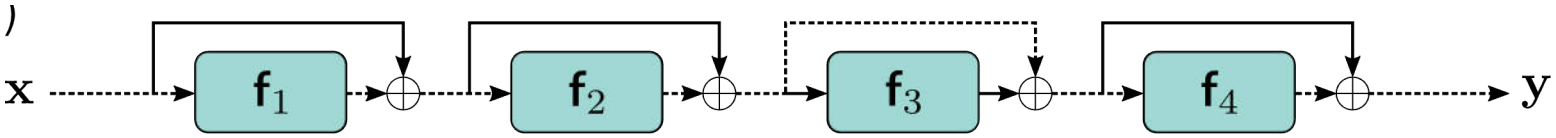

FAILS
LEARNS

Reminder: Residual Networks

Regular network: process data sequentially

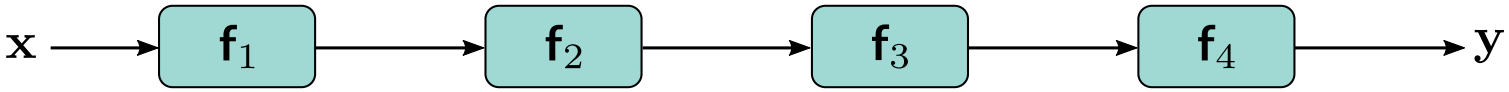


Residual Network: add skip connections

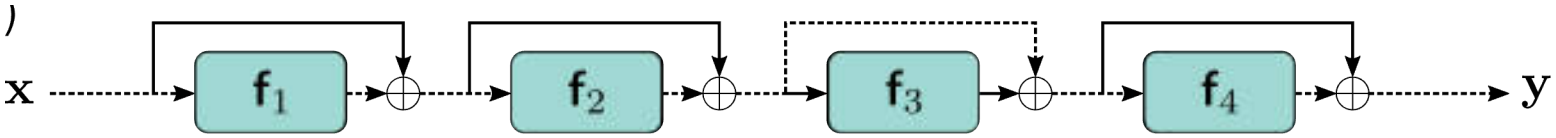


Reminder: Residual Networks

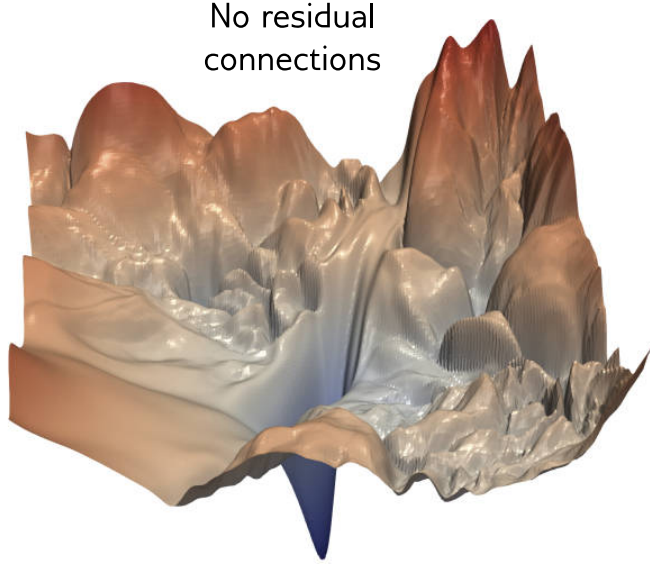
Regular network: process data sequentially



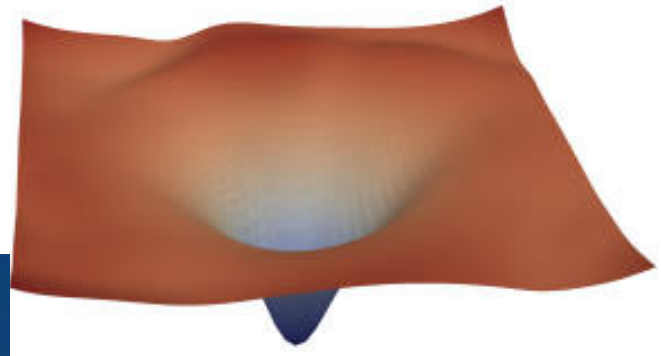
Residual Network: add skip connections



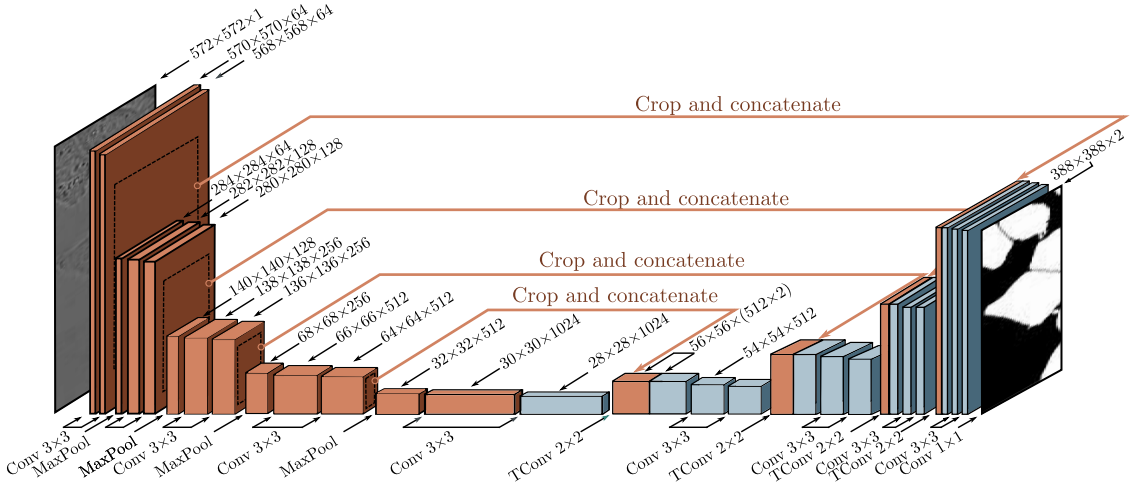
No residual connections

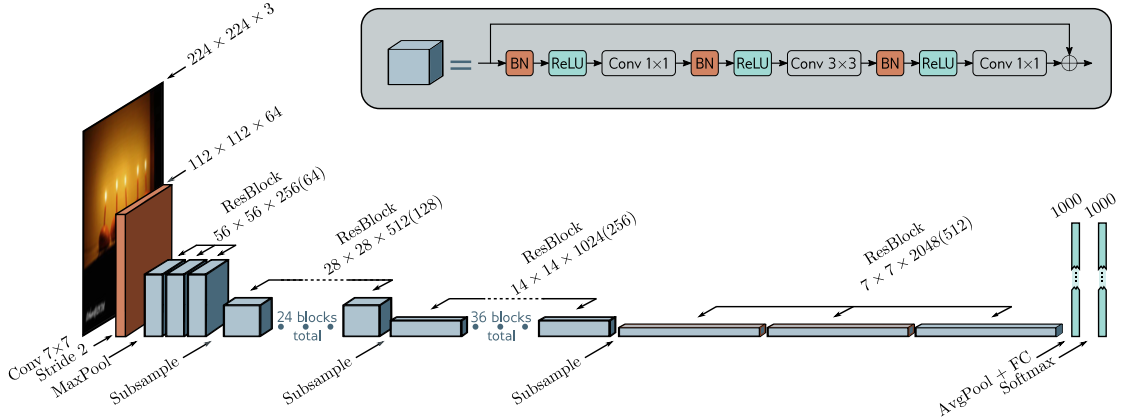


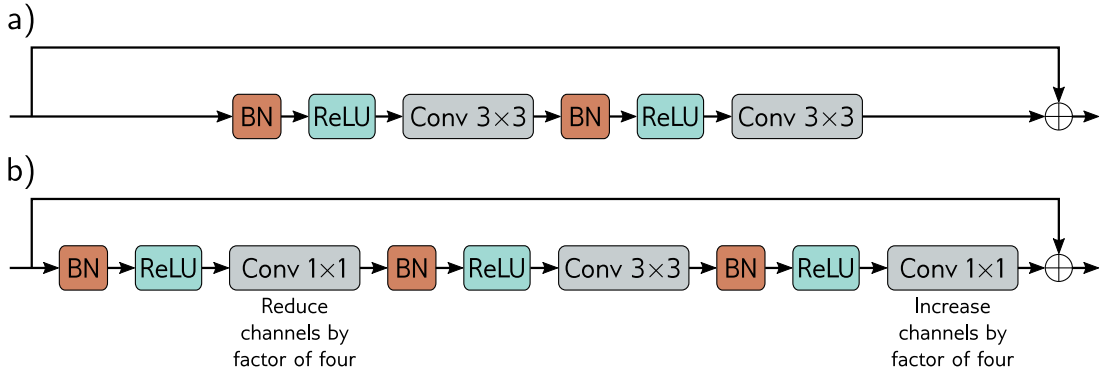
Residual connections



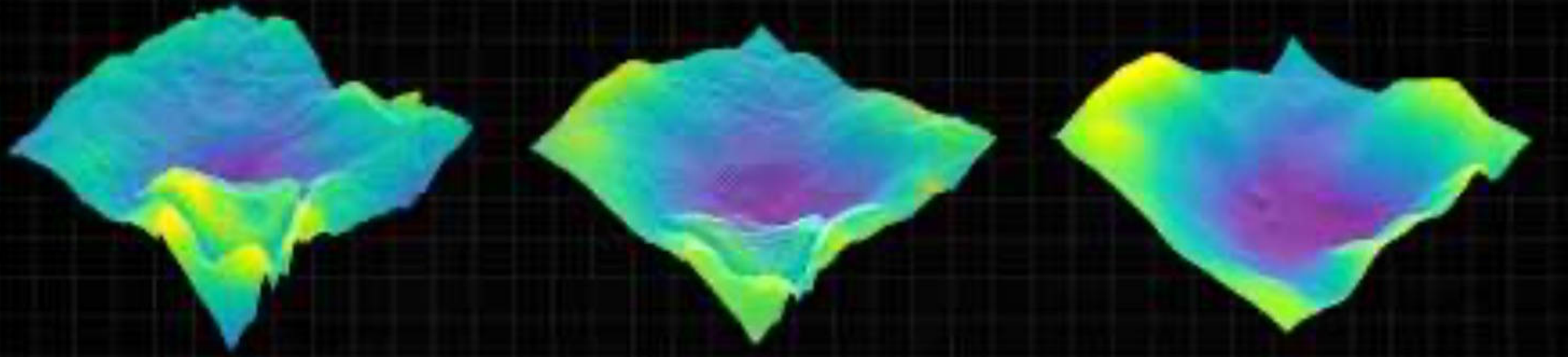
Reminder: Residual Networks



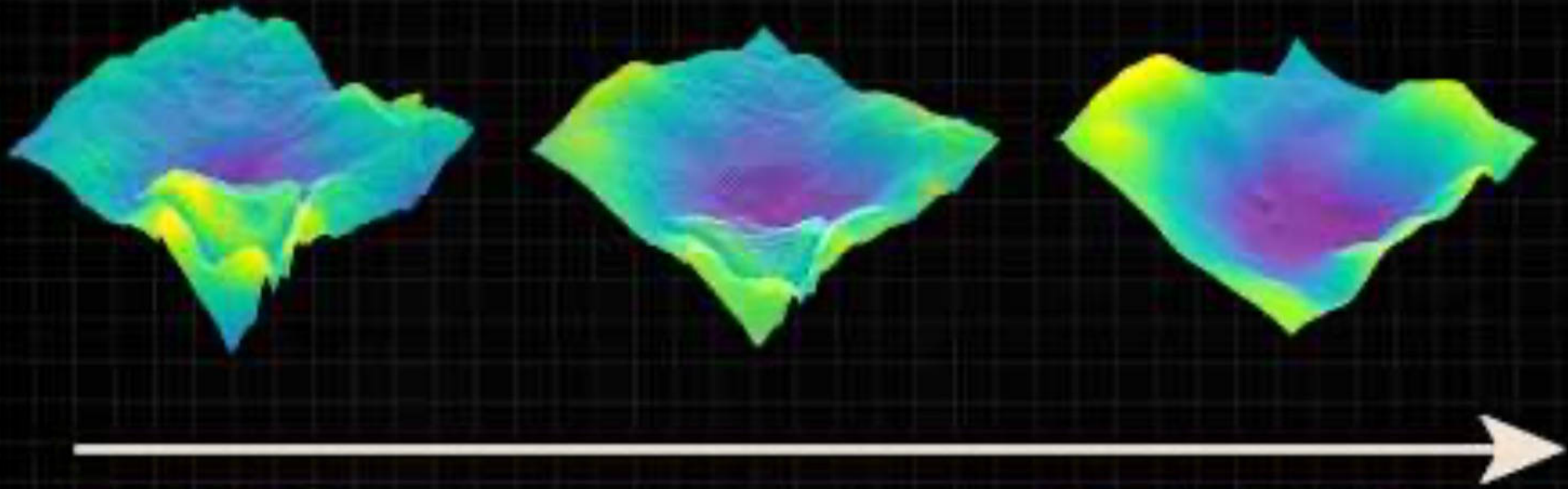




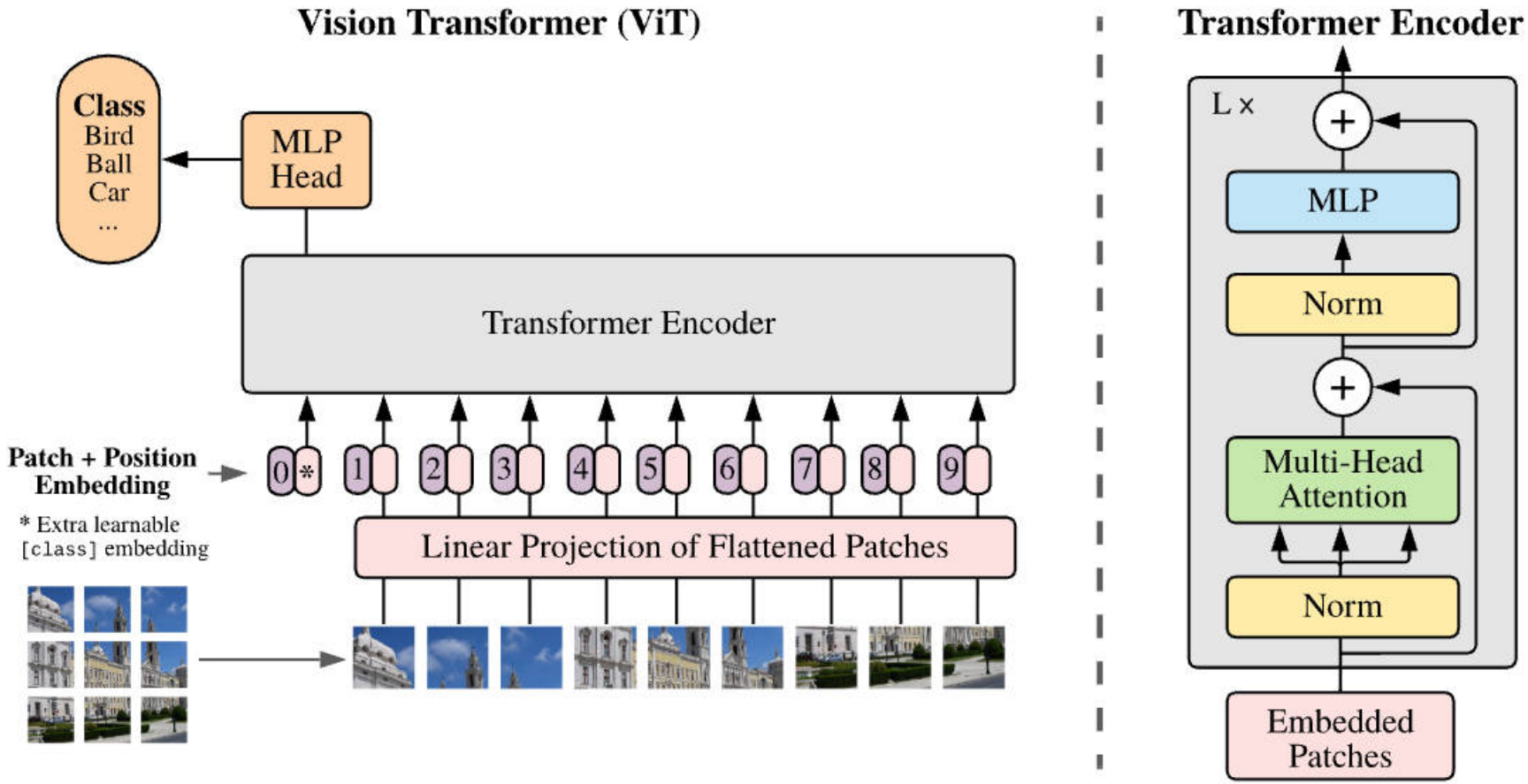
CRICOS No.00213J



FAILS LEARNS


FAILS
LEARNS

Reminder: Vision Transformers



CRICOS No.00213J

Vision Transformers



General Idea:

- Break image into patches
- Turn patches into embedding vectors
- Update embeddings based on information of the other patches (attention)
- Repeat
- Aggregate information from all patches
- Make task-specific prediction

CRICOS No.00213J

A patch starts with only a local view

The wheel patch sees texture and shape, but not yet the whole object.



- Break the image into patches.
- Project each patch via a learned linear transformation into a patch embedding vector



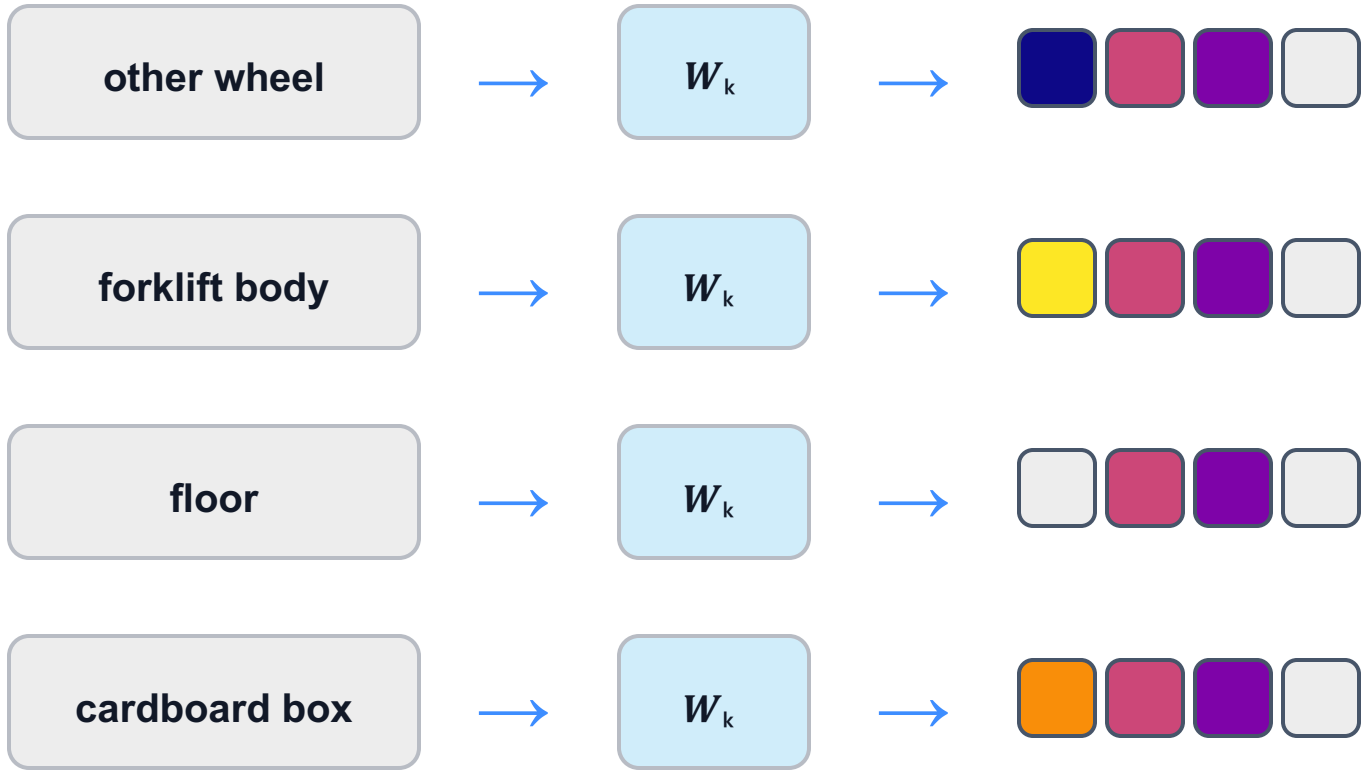
“Dark. Curved. Rubber-like. But what am I part of?”

Each patch is already a vector, but its meaning is incomplete.

CRICOS No.00213J

Every patch advertises a key

A key says: this is the kind of information I contain.

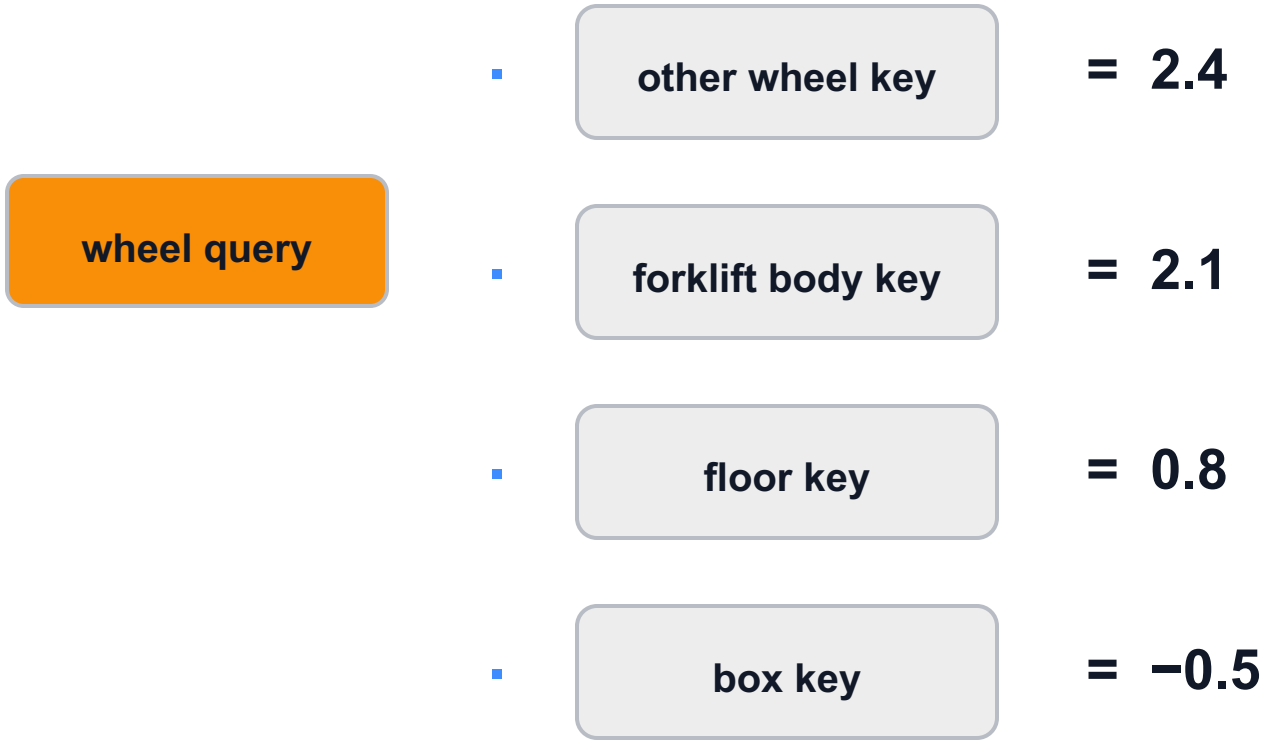


Keys are for matching – not yet the information that will be exchanged.

CRICOS No.00213J

The query meets every key

A dot product produces one learned compatibility score per patch.



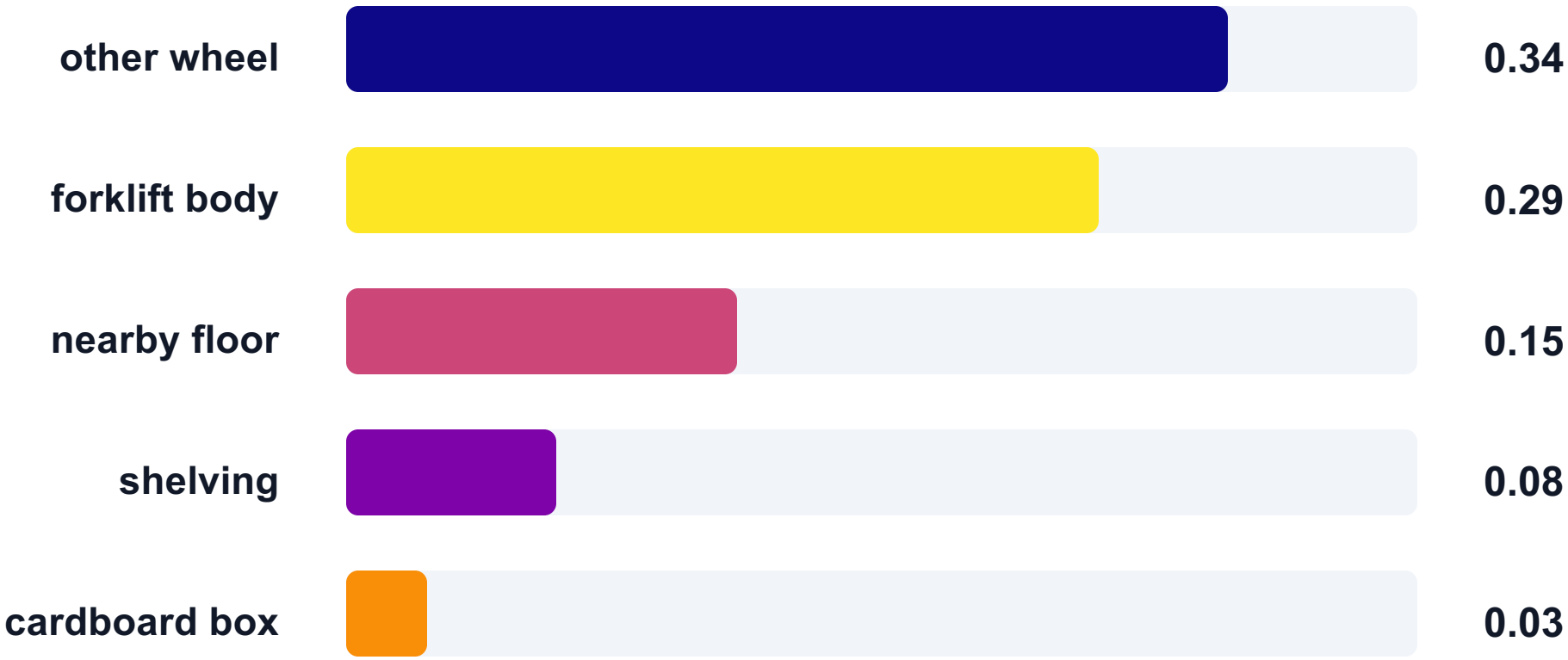
High score = useful match

Not necessarily visually identical – just relevant to the update.

CRICOS No.00213J

Softmax turns scores into attention

The wheel patch decides how much to listen to each candidate.

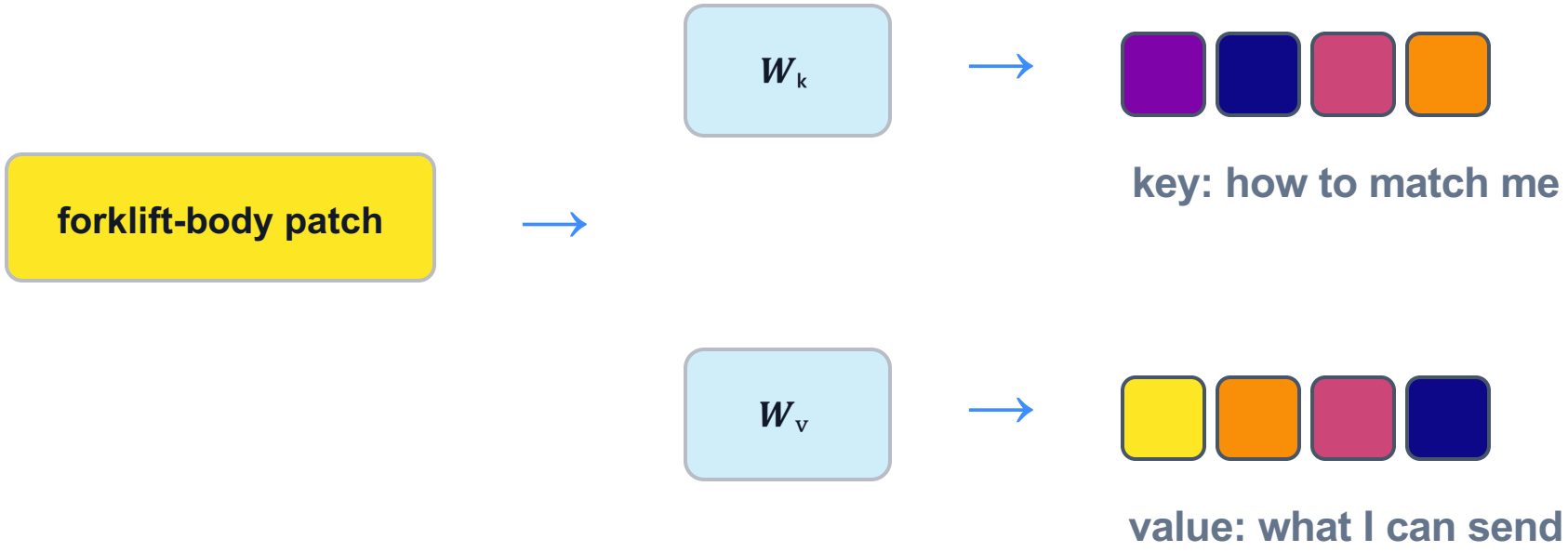


attention weights sum to 1

CRICOS No.00213J

Keys choose the source; values carry the message

Each candidate patch prepares a separate value vector to contribute.

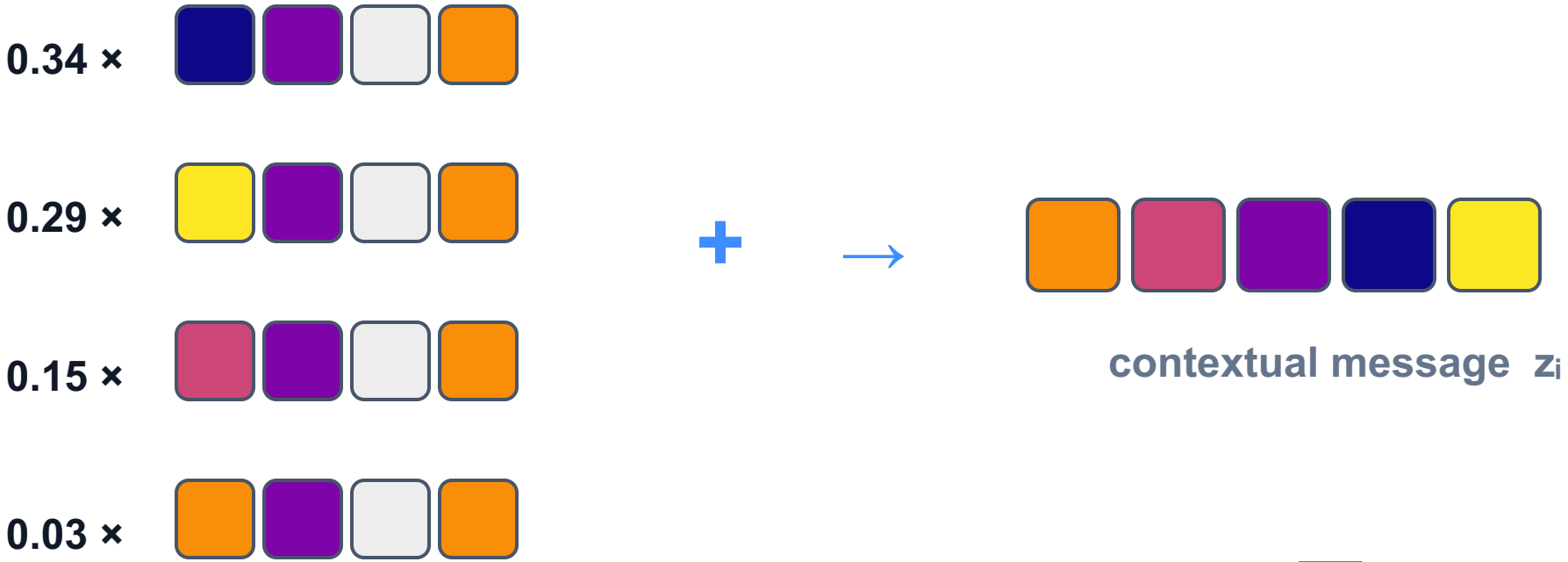


Matching representation ≠ message representation

CRICOS No.00213J

The wheel patch collects a weighted message

Values from highly attended patches contribute the most.



Attention scores × value vectors

$$z_i = \sum a_{ij} v_j$$

The patch keeps what it knew and adds context

A residual connection protects the existing representation.



original x_i attention message z_i updated wheel token x'_i

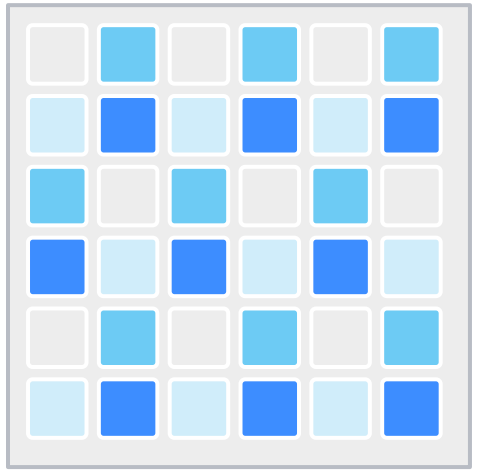
“I still know what I saw locally, but now I also know the context.”

CRICOS No.00213J

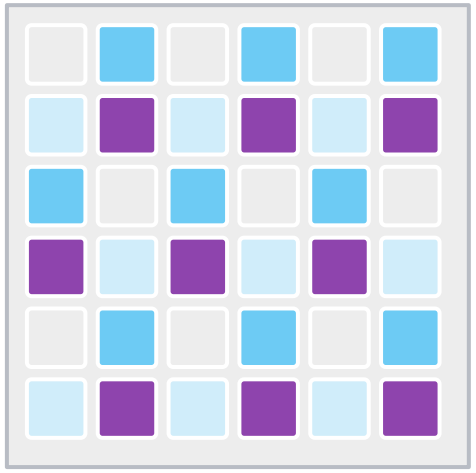
Every patch runs its own conversation

Stack queries and keys into matrices; transpose K so every query can compare with every key.

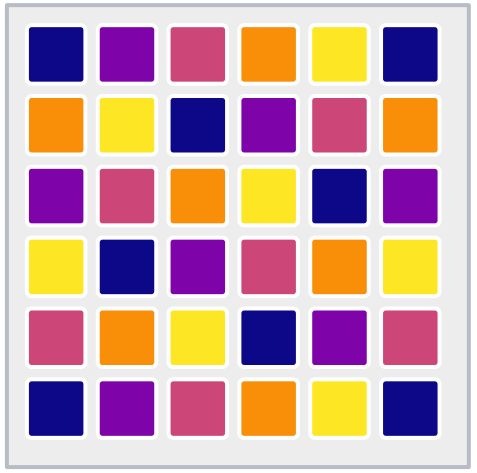
Q



K



S



Queries: one q_i per row

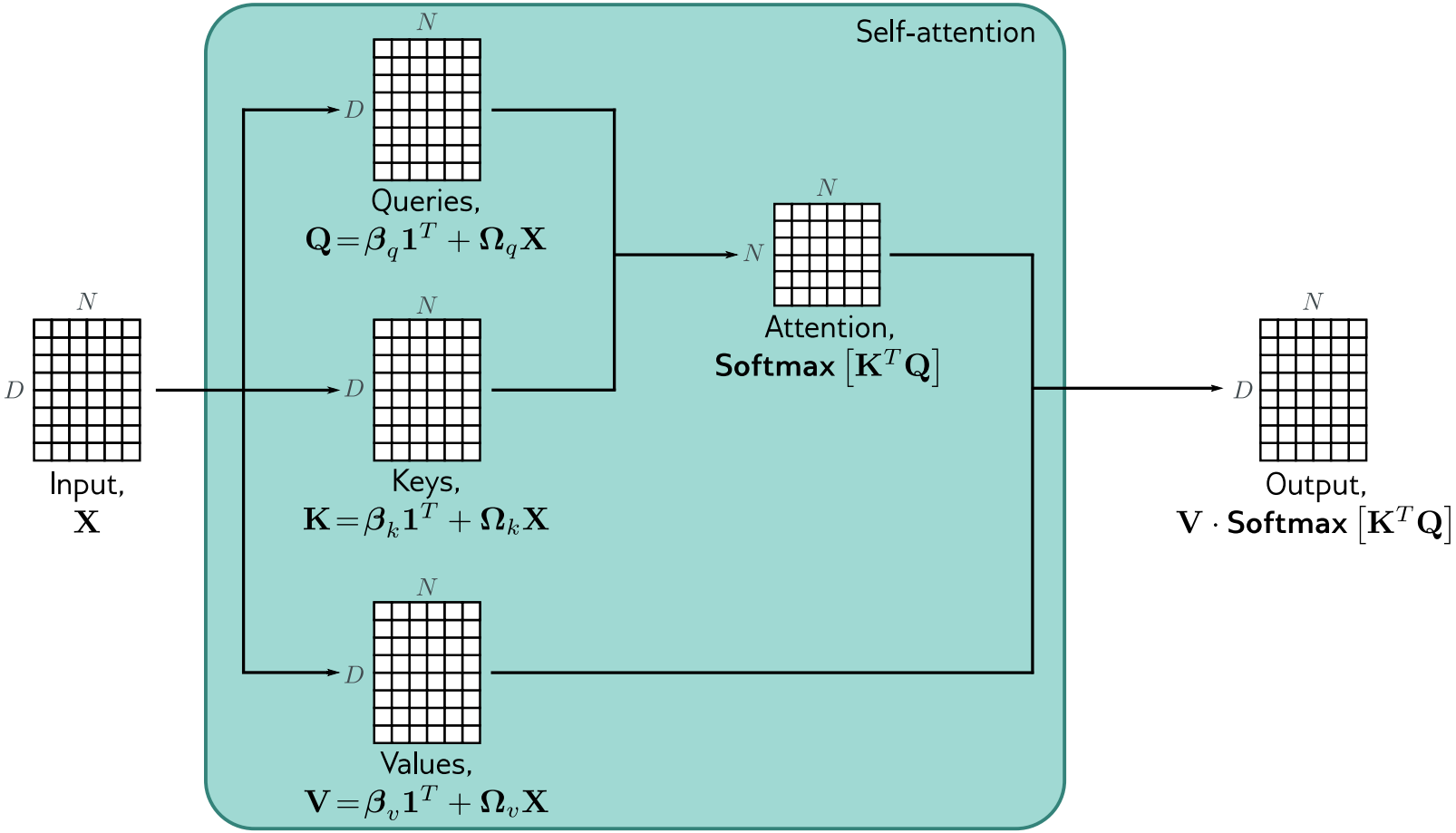
Keys: one k_j per row

transpose K → K^T for multiplication

Attention scores: $A_{ij} = q_i \cdot k_j$

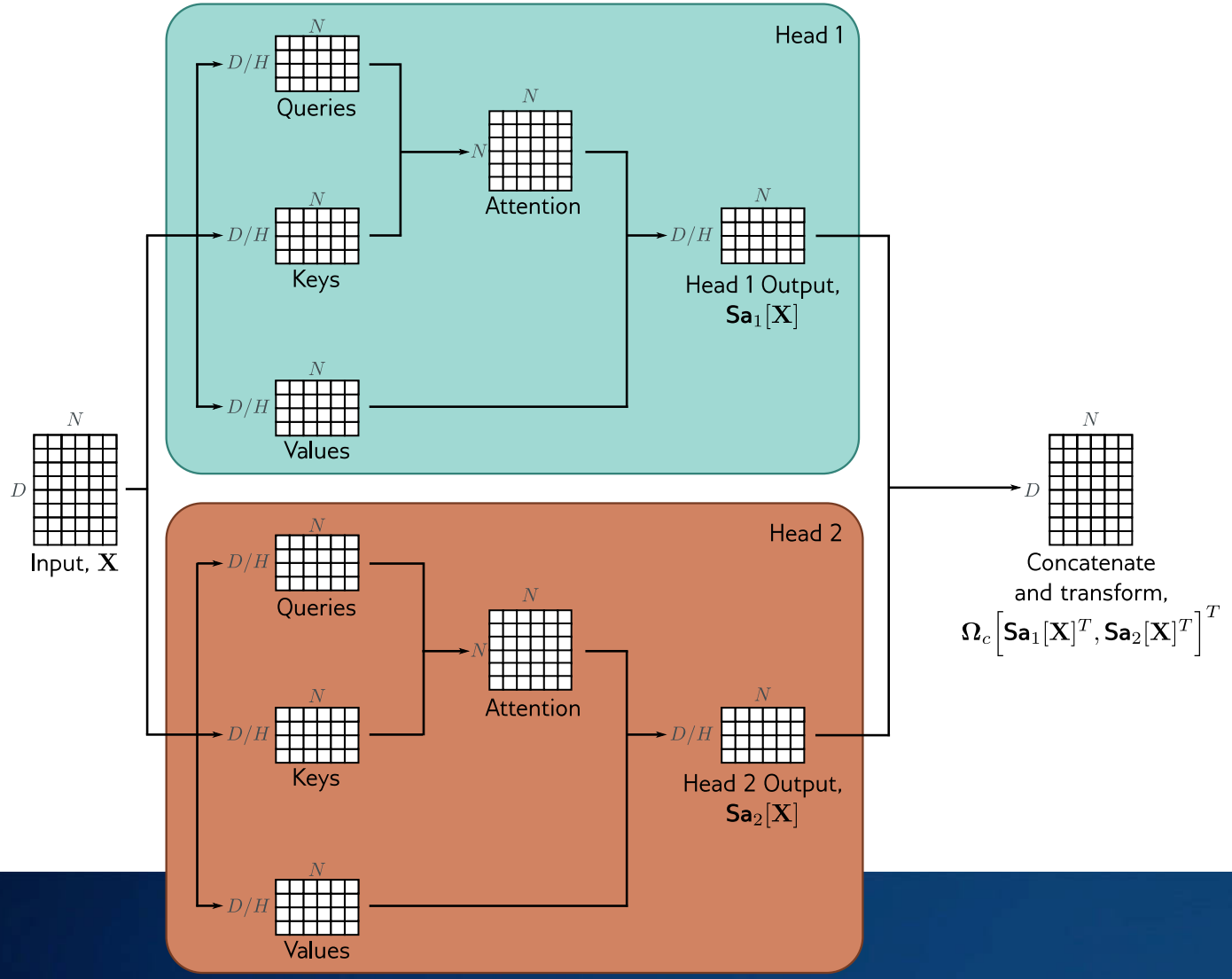
$$Q \cdot K^T \rightarrow \text{every query–key comparison}$$

One Attention Block

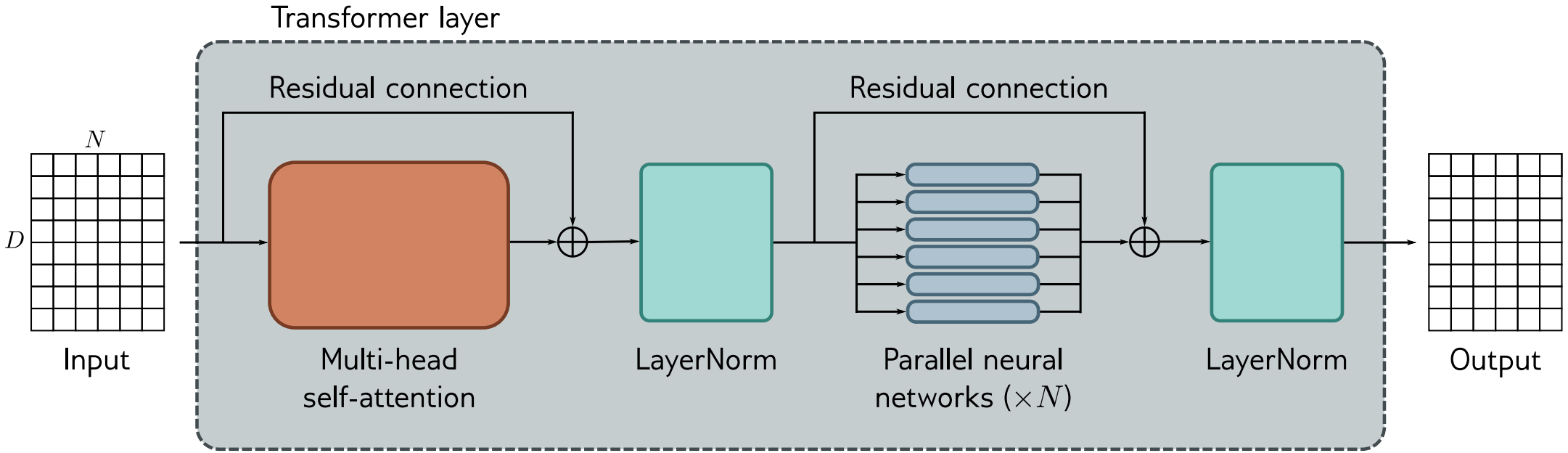


CRICOS No.00213J

Multi-Head Attention

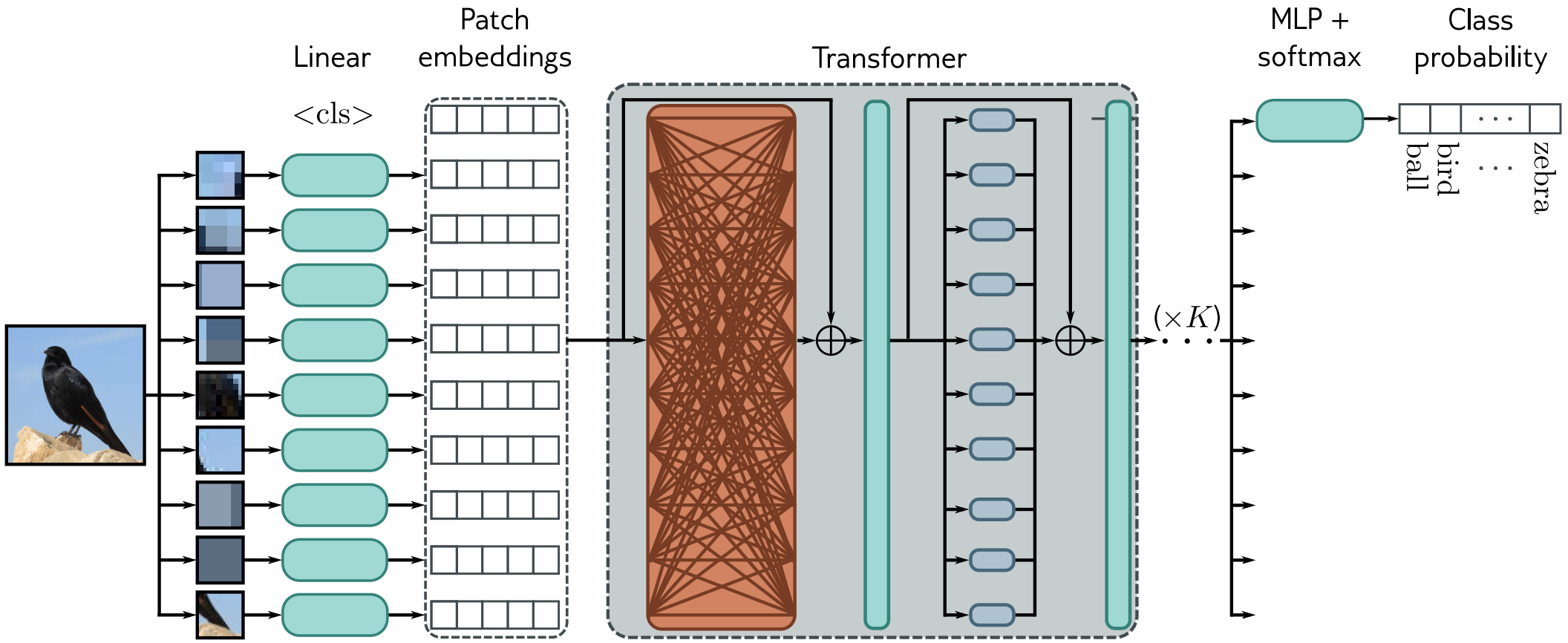


One Transformer Layer



CRICOS No.00213J

Combine Multiple Transformer Layers

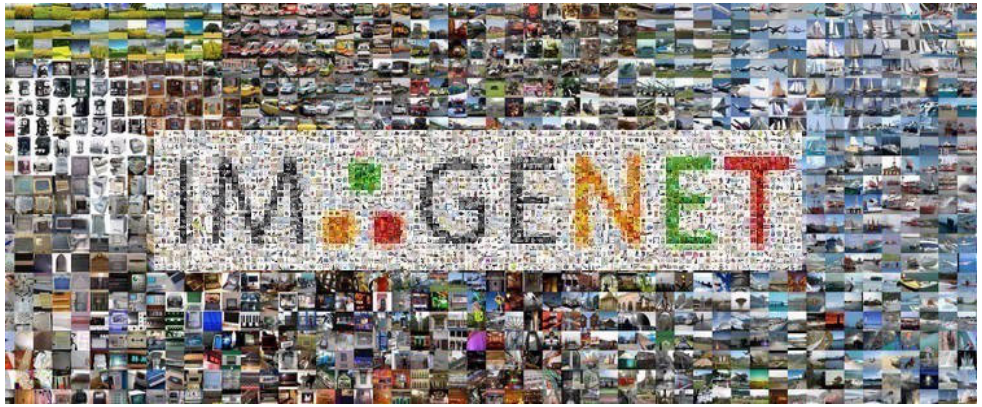


CRICOS No.00213J

Illustration by Simon Prince, Understanding Deep Learning.

From Computer Vision to Robotic Vision

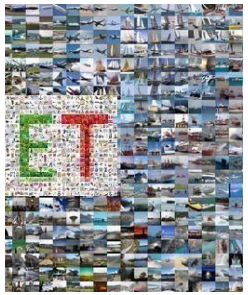
Computer Vision "tests" model performance on data



Training



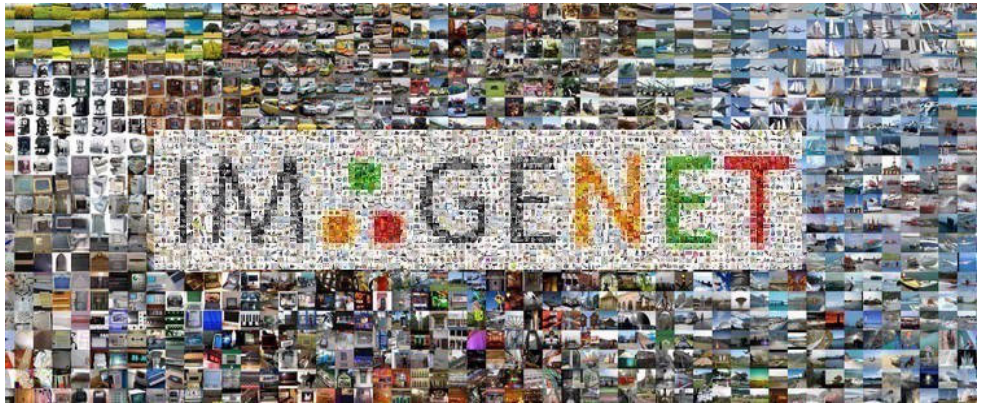
Evaluating



Performance

87%
Accuracy

Robotic Vision "deploys" the model into the world



Training



Evaluating





Performance

87%
Accuracy

Deploying in the world requires understanding the "cost" of errors



vs.



From scores to decisions

A model produces scores; an operating threshold converts scores into actions.

Lower thresholds → more positive predictions → typically higher recall, more false positives.

Higher thresholds → fewer positive predictions → typically higher precision, more false negatives.

The right trade-off depends on physical consequences, not leaderboard convention.

Low threshold

↑ Recall ↑ False positives

High threshold

↑ Precision ↑ False negatives

Exact behaviour depends on your data distribution.

Asymmetric error costs

	Predicted: OBSTACLE	Predicted: CLEAR
Actual: OBSTACLE	**True Positive** Obstacle detected correctly. Robot stops safely.	**False Positive** Unnecessary stop. Mission delay; low physical risk.
Actual: CLEAR	**False Negative** Missed obstacle. Potential collision — high cost.	**True Negative** Clear corridor confirmed. Robot proceeds.

CRICOS No.00213J

Confidence is not guaranteed probability

A softmax score expresses relative model preference — not a calibrated probability.

A score of 0.9 does not mean 90% of such predictions are correct.

Calibration asks: does reported confidence match observed correctness frequency?

Confidence can remain high even under significant distribution shift.

⚠ Overconfident model

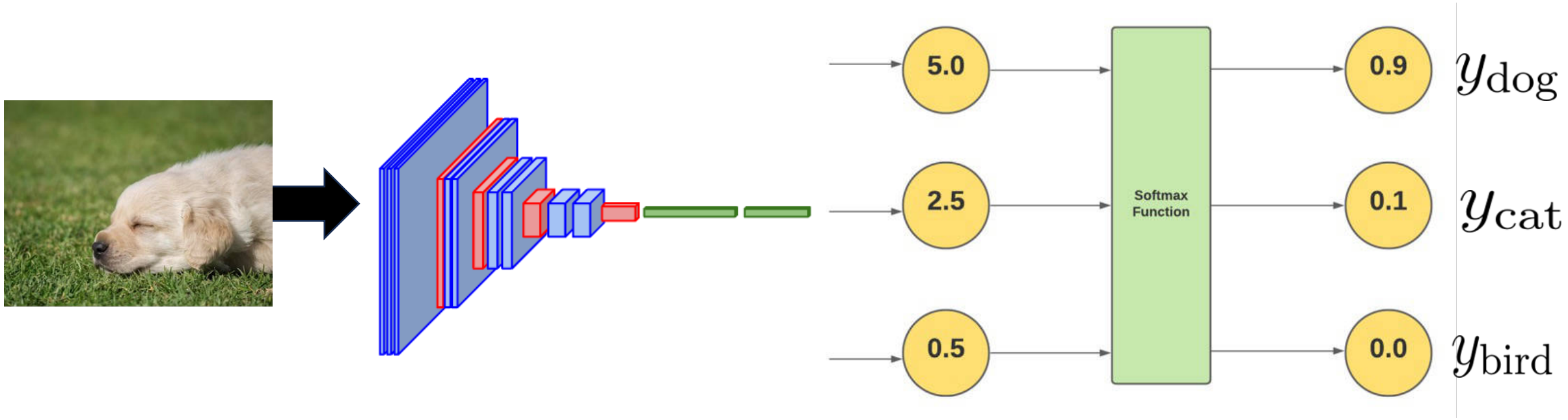
High score on test set.
Distribution shifts.
Score stays high.
Robot acts on wrong prediction.

Do not describe softmax output as calibrated probability unless calibration has been assessed.

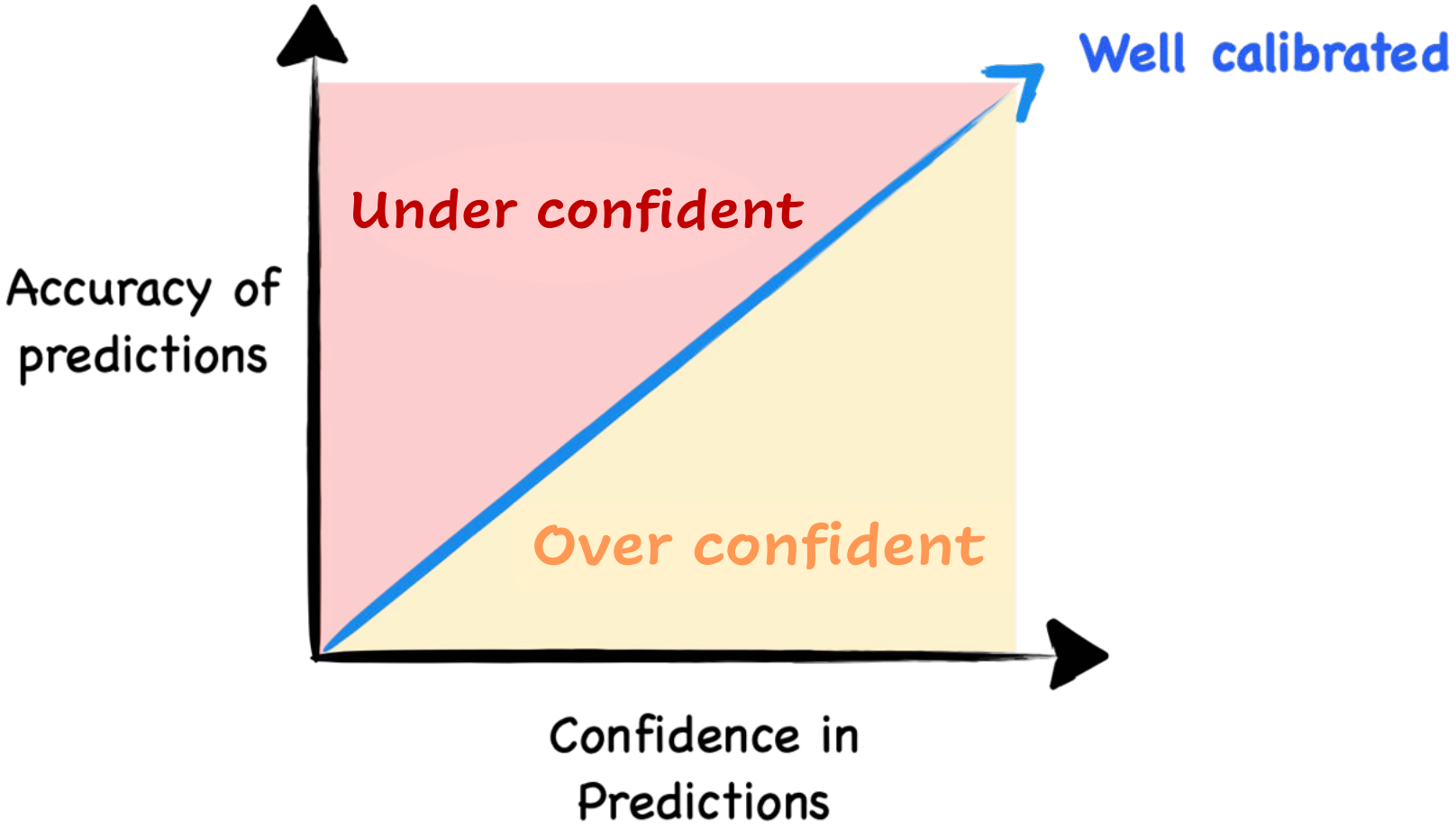
CRICOS No.00213J

Deep Neural Classifiers give a prediction "confidence"

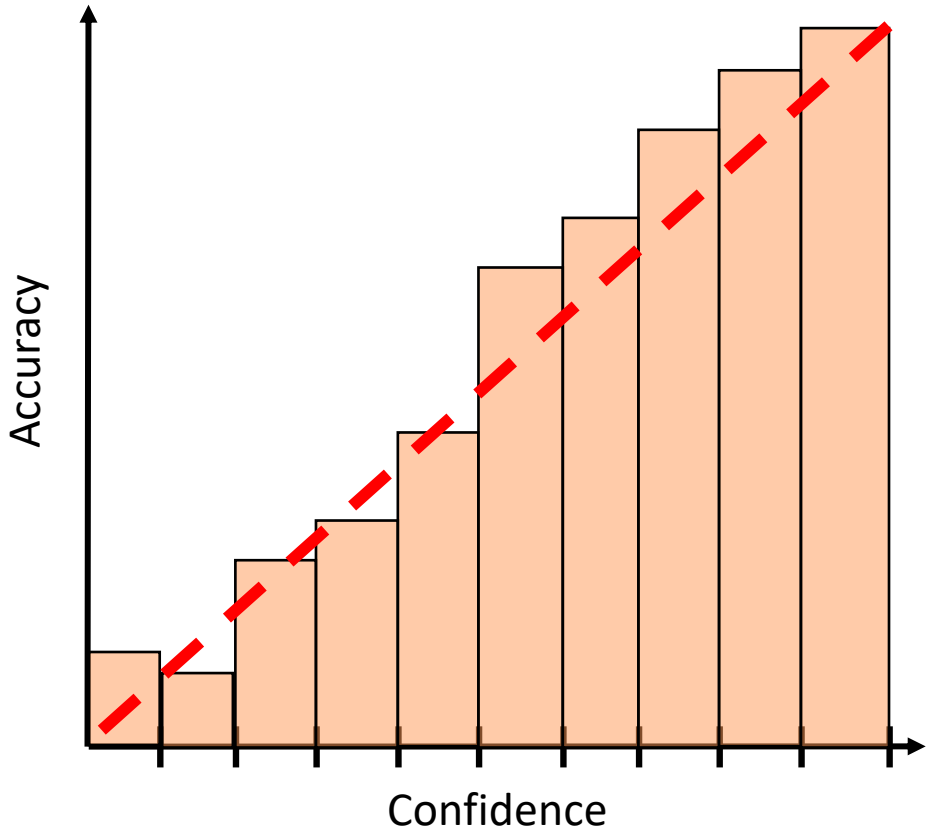
"90% confident the input belongs to class *dog*".



Confidence Calibration



Measuring Confidence Calibration

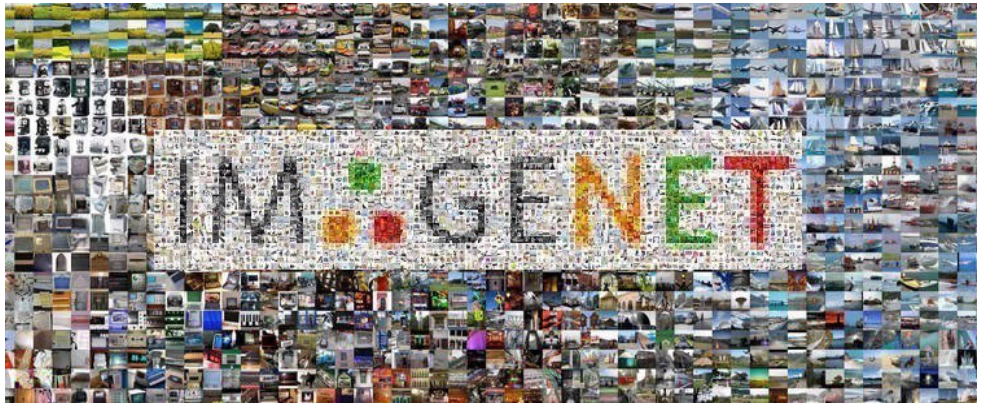


1. Test the model on the dataset, and for every prediction store:
    1. If it was correct or incorrect
    2. Model confidence

2. Divide all the possible confidence values into N bins (e.g. 10 bins = 0-10%, 10-20%, 20-30%, …., 80-90%, 90-100%).

3. For each bin:
    1. Find the predictions in that confidence range (e.g. 0-10%)
    2. Calculate the mean accuracy of all those predictions
    3. Record on the plot

Robotic Vision "deploys" the model into the world



Training



Evaluating



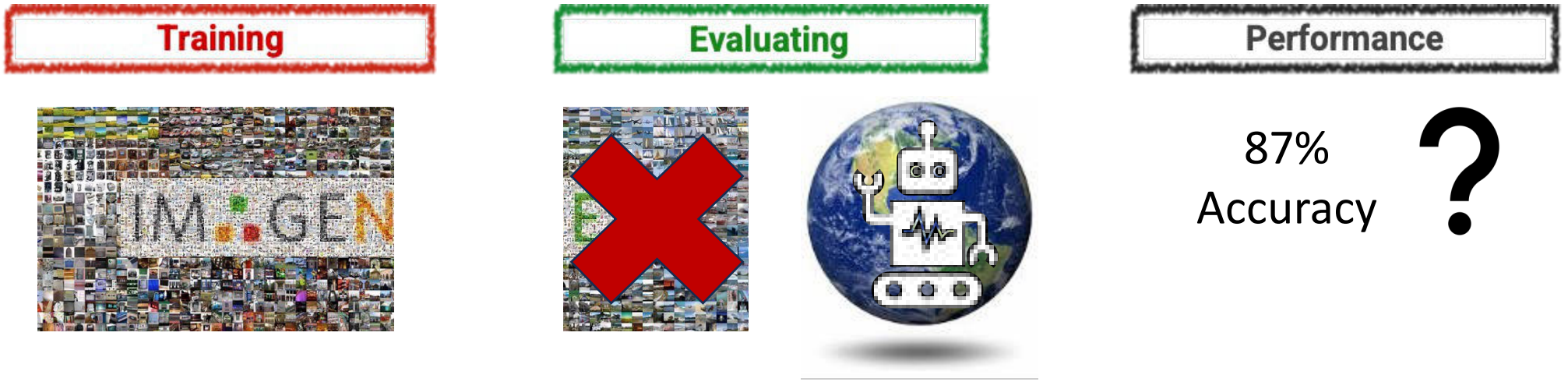


Performance

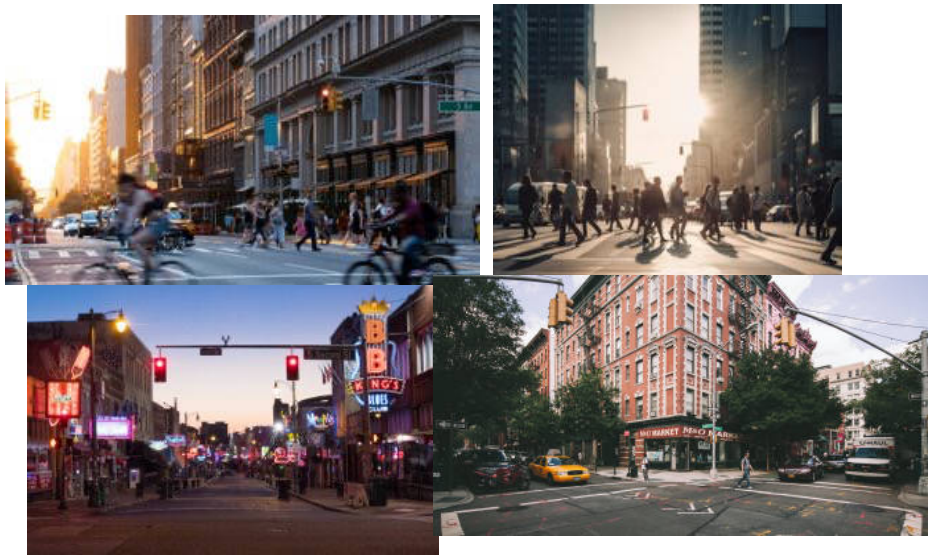
87%
Accuracy

A major assumption in Deep Learning…

The dataset distribution matches the distribution of the data encountered in the end application of the model.



Distribution Shift -- Feature Shift

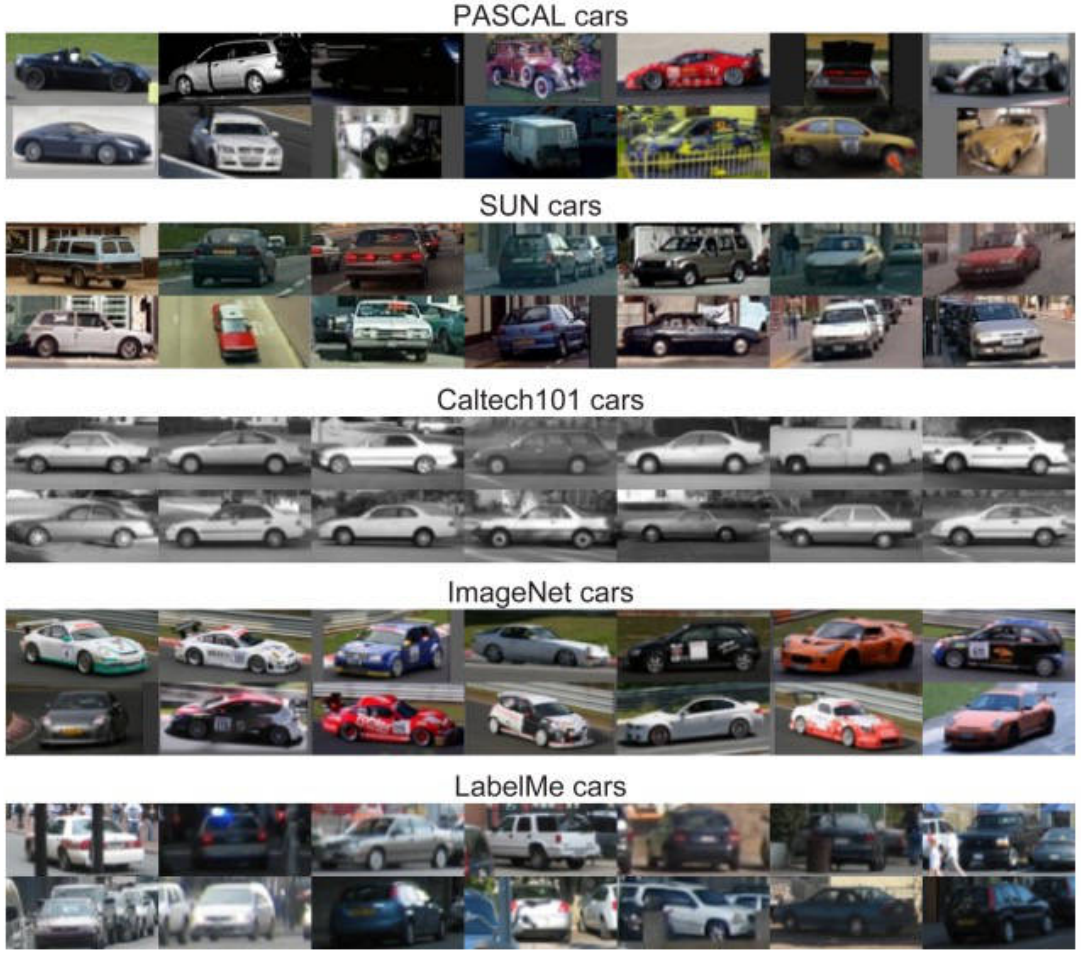


Training data



Changes to features

Deep Learning Robustness to Feature Shift



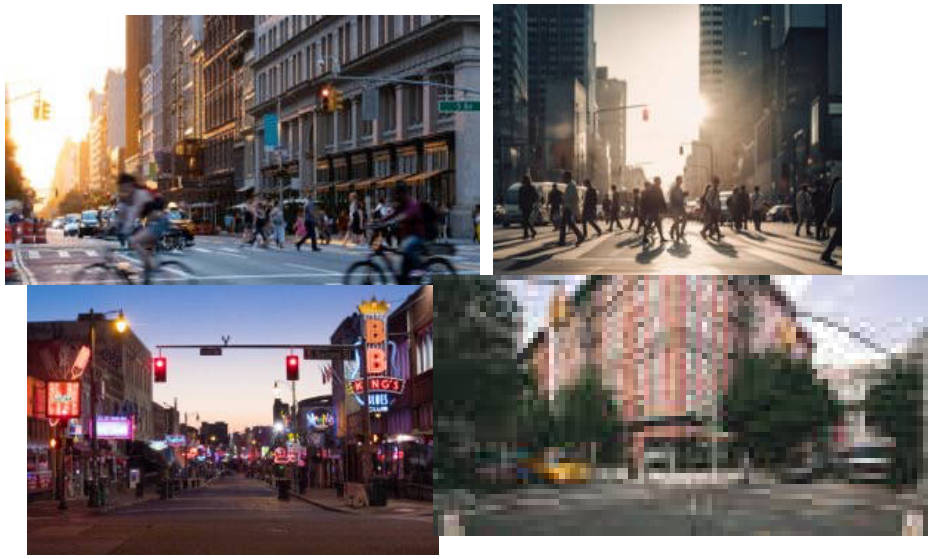
Unbiased Look at Dataset Bias

Antonio Torralba
Massachusetts Institute of Technology
torralba@csail.mit.edu

Alexei A. Efros
Carnegie Mellon University
efros@cs.cmu.edu

task	Test on: / Train on:	Self	Mean others	Percent drop
"car" classification	SUN09	28.2	19.8	**30%**
	LabelMe	34.0	24.5	**28%**
	PASCAL	35.2	32.6	**7%**
	ImageNet	57.4	34.4	**40%**
	Caltech101	96.9	26.7	**73%**
	MSRC	68.4	26.8	**61%**
	Mean others	53.4	27.5	48%

Distribution Shift -- Class Shift



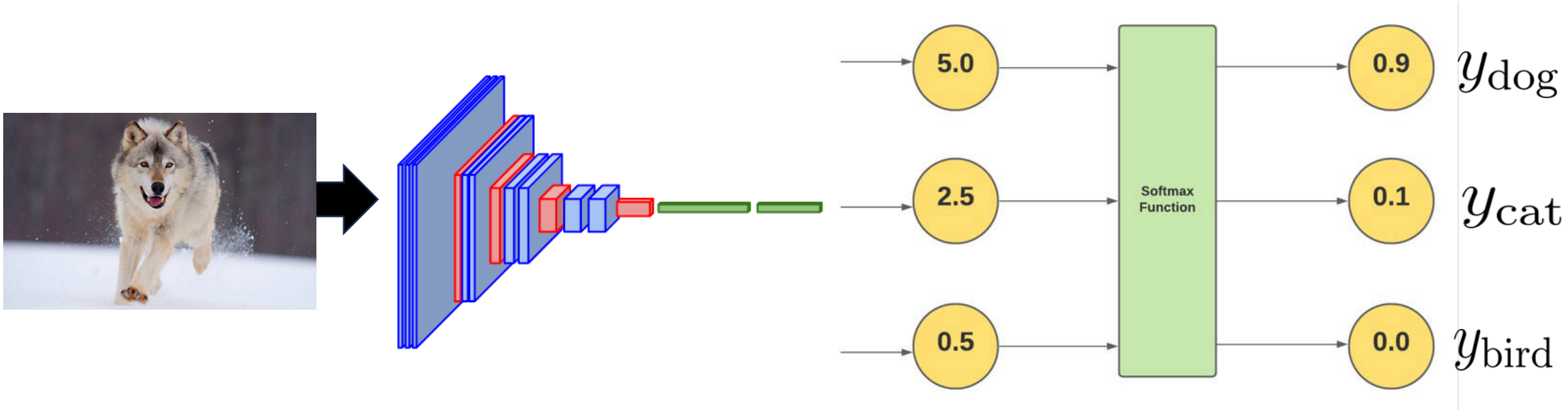
Training data



Changes to classes

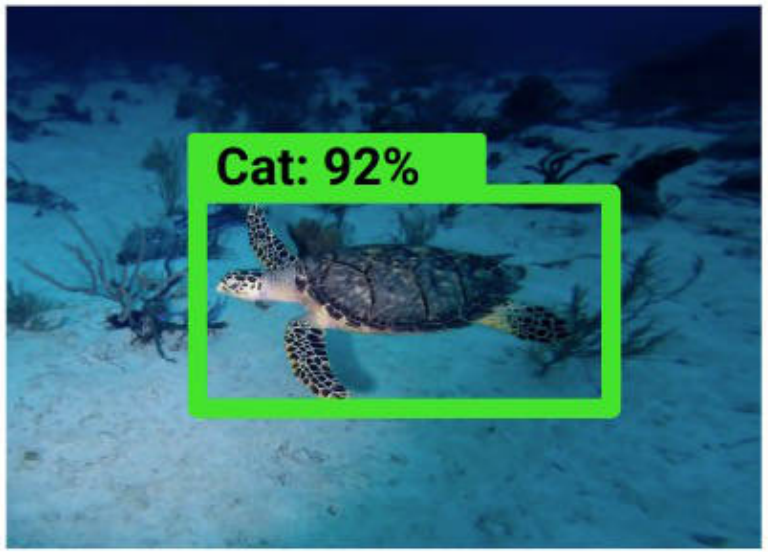
Robustness to Class Shift and New Classes

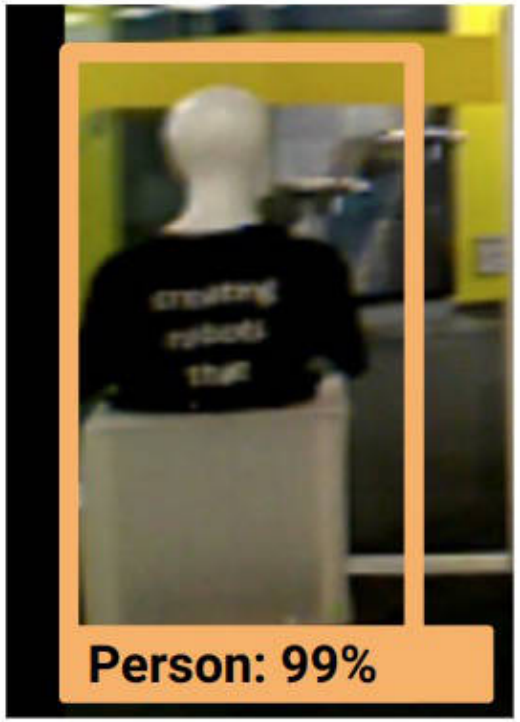
Most CNNs don't have a clear mechanism for handling new classes (classes not seen during training).



Robustness to Distribution Shift: New Classes

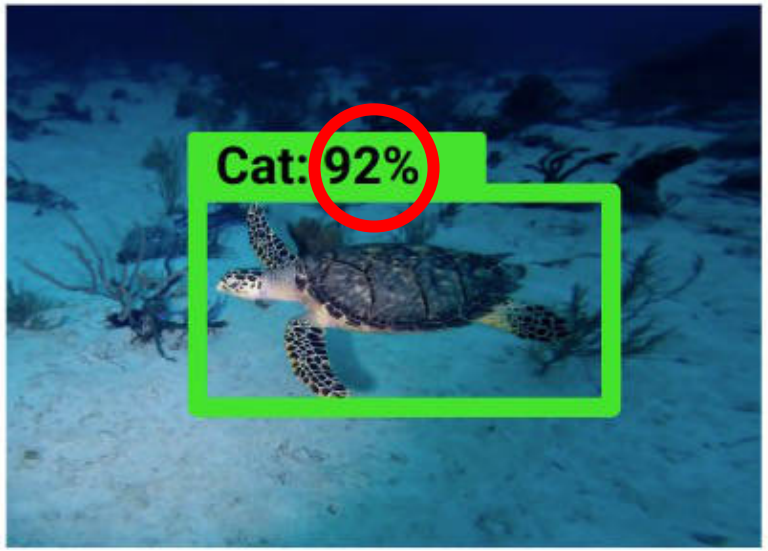


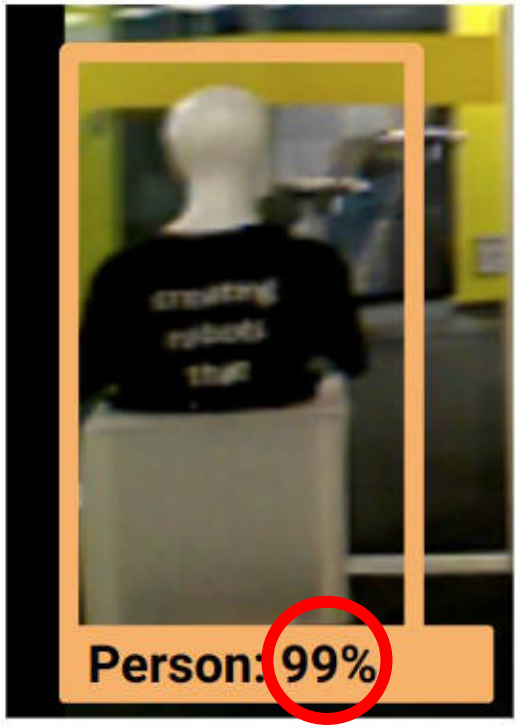




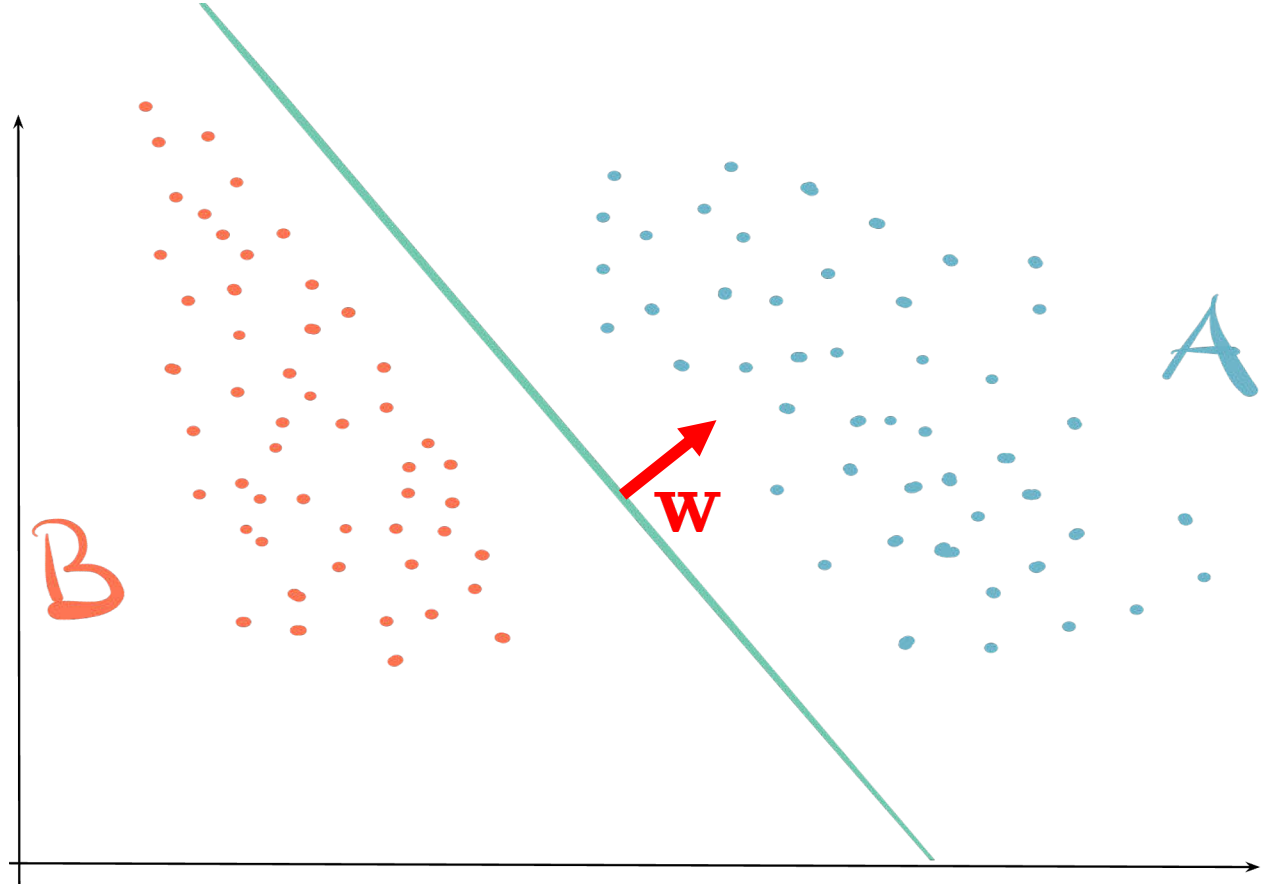
Neural networks often misclassify new classes with high confidence





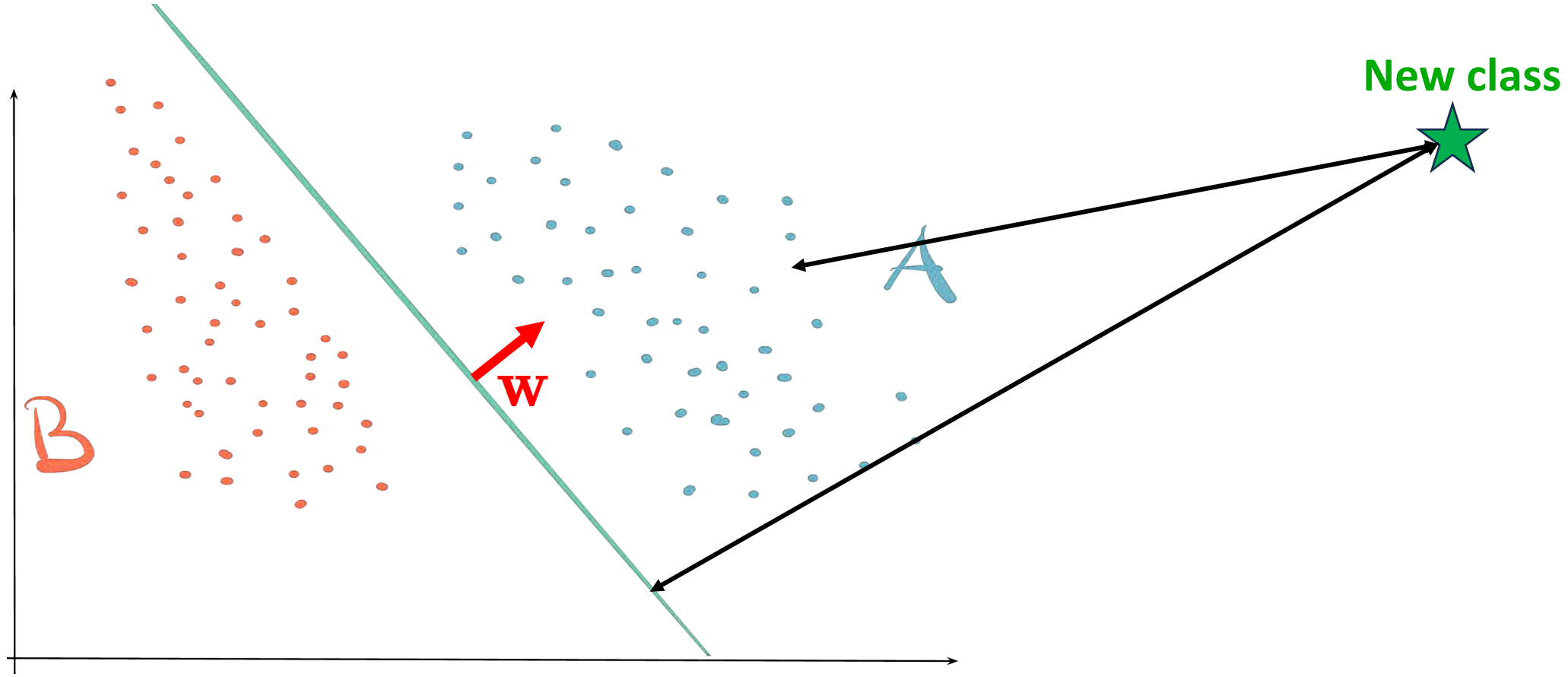


Neural networks often misclassify new classes with high confidence: Why?



The class score is proportional to the distance from the decision boundary.

Neural networks often misclassify new classes with high confidence: Why?

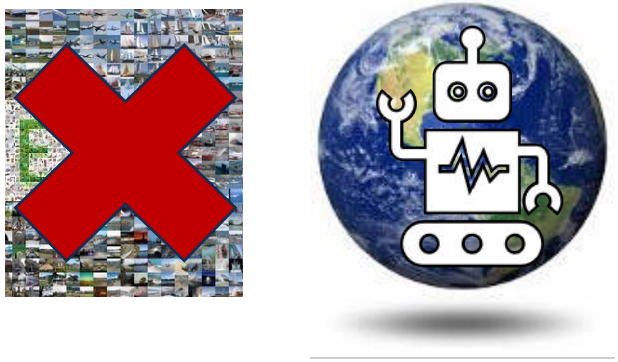


How should we use deep learning for robotic vision?

Training



Evaluating



Performance

87%
Accuracy

?

Questions

Next Week: Learned geometry and modern 3D scene representations